



RU mine (1)

2021

1 (a) 2

▼ What is the benefit of binary search over linear search?

Binary search has several advantages over linear search:

1. **Efficiency:** Binary search is significantly more efficient than linear search, especially for large data sets. In binary search, the search space is divided in half at each step, allowing for a much faster convergence toward the desired element. As a result, binary search has a time complexity of $O(\log n)$, whereas linear search has a time complexity of $O(n)$, where n is the number of elements in the data set.
2. **Reduced Comparison:** Binary search eliminates a large number of comparisons compared to linear search. In linear search, each element in the data set needs to be compared with the target element until a match is found. In binary search, however, the search space is continuously divided in half, reducing the number of elements to be compared. This makes binary search especially advantageous when dealing with large data sets.
3. **Sorted Data Requirement:** Binary search requires the data set to be sorted in ascending or descending order. On the other hand, linear search does not have any specific requirements regarding the data order, which can be both an advantage and a disadvantage depending on the context.
4. **Better Worst-Case Performance:** Binary search guarantees a worst-case time complexity of $O(\log n)$, which means that even in the worst-case scenario, the search can be completed much faster compared to linear search. In linear search, the worst-case scenario occurs when the target element is at the end of the data set, requiring a full traversal of all elements.

1 (b) 3.75

▼ Maze(1:6, -4:1, 5:10) is a 3D array with base=100, w=4, calculate Maze [3,-2,8] address is a row major order and column major order #revised-1

1. Row Major Order:

To find the address of the element using row-major order, use the following formula:

$$\text{Address of } A[i][j][k] = B + W * (M * N(i-x) + N * (j-y) + (k-z))$$

Here:

B = Base Address (start address)

W = Weight (storage size of one element stored in the array)

M = Row (total number of rows)

N = Column (total number of columns)

P = Width (total number of cells depth-wise)

x = Lower Bound of Row

y = Lower Bound of Column

z = Lower Bound of Width

Example: Given an array, `arr[1:9, -4:1, 5:10]` with a base value of **400** and the size of each element is **2 Bytes** in memory find the address of element `arr[5][-1][8]` with the help of row-major order?

Solution:

Given:

Row Subset of an element whose address to be found $I = 5$

Column Subset of an element whose address to be found $J = -1$

Block Subset of an element whose address to be found $K = 8$

Base address $B = 400$

Storage size of one element store in any array(in Byte) $W = 2$

Lower Limit of row/start row index of matrix $x = 1$

Lower Limit of column/start column index of matrix $y = -4$

Lower Limit of blocks in matrix $z = 5$

$M(\text{row}) = \text{Upper Bound} - \text{Lower Bound} + 1 = 9 - 1 + 1 = 9$

$N(\text{Column}) = \text{Upper Bound} - \text{Lower Bound} + 1 = 1 - (-4) + 1 = 6$

Formula used:

Address of `arr[I][J][K]` $= B + W (M * N(i-x) + N * (j-y) + (k-z))$

Solution:

Address of `arr[5][-1][8]` $= 400 + 2 * \{[9 * 6 * (5 - 1)] + 6 * [(-1 + 4)]\} + [8 - 5]$

$$= 400 + 2 * (9*6*4) + (6*3) + 3$$

$$= 400 + 2 * (237)$$

$$= 874$$

2. Column Major Order:

To find the address of the element using column-major order, use the following formula:1

$$\text{Address of } A[i][j][k] = B + W(M * N(i - x) + M * (k - z) + (j - y))$$

Here:

B = Base Address (start address)

W = Weight (storage size of one element stored in the array)

M = Row (total number of rows)

N = Column (total number of columns)

P = Width (total number of cells depth-wise)

x = Lower Bound of Row

y = Lower Bound of Column

z = Lower Bound of Width

Example: Given an array `arr[1:8, -5:5, -10:5]` with a base value of **400** and the size of each element is **4 Bytes** in memory find the address of element `arr[3][3][3]` with the help of column-major order?

Solution:

Given:

Row Subset of an element whose address to be found $I = 3$

Column Subset of an element whose address to be found $J = 3$

Block Subset of an element whose address to be found $K = 3$

Base address $B = 400$

Storage size of one element store in any array(in Byte) $W = 4$

Lower Limit of row/start row index of matrix $x = 1$

Lower Limit of column/start column index of matrix $y = -5$

Lower Limit of blocks in matrix $z = -10$

M (row)= Upper Bound – Lower Bound + 1 = $8-1+1 = 8$

N (column)= Upper Bound – Lower Bound + 1 = $5+5+1 = 11$

Formula used:

*Address of $[i][j][k] = B + W(M * N(i - x) + M * (j - y) + (k - z))$*

Solution:

$$\begin{aligned}
 \text{Address of } arr[3][3][3] &= 400 + 4 * ((8*11*(3-1)+8*(3-(-5))+(3-(-10)))) \\
 &= 400 + 4 * ((88*2 + 8*8+13)) \\
 &= 400 + 4 * (253) \\
 &= 400 + 1012 \\
 &= 1412
 \end{aligned}$$

1 (c) 3

▼ We often store sparse matrix in a 1D array to save spaces. What is the memory saving if we store a sparse matrix in a 1D array rather than a 2D array? #2017

Storing a sparse matrix in a 1D array can result in significant memory savings compared to using a 2D array.

In a 2D array, memory is allocated for each element, regardless of its value. This means that even if the matrix is mostly composed of zeros, memory is still consumed for those zero values. On the other hand, a 1D array representation of a sparse matrix only stores the non-zero elements, along with their corresponding indices or positions.

By eliminating the need to store zero elements explicitly, the memory requirements of the sparse matrix decrease. This can result in significant memory savings, especially for large matrices with high sparsity. The more sparse the matrix is, the more memory can be saved by using a 1D array representation.

2 (a) 2.75

▼ What is a linked list? Explain the main difference between the linked list and linear array? #2020

A linked list is a data structure that consists of a sequence of nodes, where each node contains both data and a reference (or link) to the next node in the sequence. It provides a flexible way to store and manipulate data dynamically.

Comparison: Linked List vs. Linear Array

Linked List	Linear Array
Dynamically allocated nodes	Contiguous memory allocation for elements
Requires additional memory for link pointers	Memory-efficient, no extra overhead for pointers
Efficient for inserting/deleting at any position	Inefficient for inserting/deleting in the middle
Sequential traversal required for accessing elements	Direct access to elements by index
Can easily accommodate variable size and growth	Fixed size, need resizing or reallocation for growth
Requires sequential search for finding an element	Allows direct access by index, faster searching
Relatively simpler implementation	Requires more complex indexing and resizing

2 (b) 3

▼ Briefly discuss the terms garbage collection, overflow and underflow

1. Garbage Collection:

Garbage collection is an automatic memory management technique used in programming languages, particularly those with dynamic memory allocation. It refers to the process of automatically reclaiming memory that is no longer in use by the program. The garbage collector identifies and frees up memory that is no longer reachable by the program, allowing it to be reused for other purposes. This helps prevent memory leaks and reduces the burden of manual memory management.

2. Overflow:

Overflow occurs when an operation results in a value that exceeds the maximum representable value for a given data type or memory space. For example, in a signed integer data type, if the result of an addition operation exceeds the maximum value that can be represented, it overflows. Overflow can lead to unexpected incorrect results in calculations.

3. Underflow:

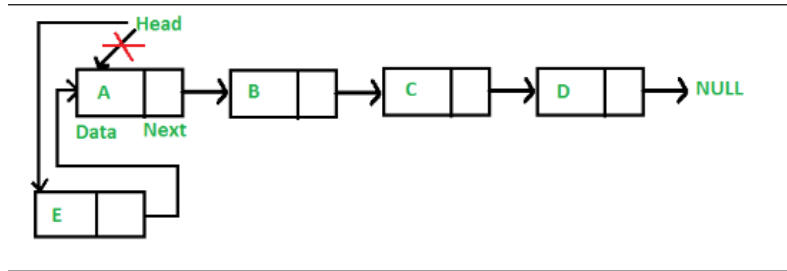
Underflow occurs when an operation results in a value that is smaller than the minimum representable value for a given data type or memory space. Similar to overflow, underflow can lead to unexpected incorrect results in calculations. Underflow occurs in scenarios such as subtracting a larger value from a smaller value. Some programming languages may handle underflow by rounding the result, or by generating an error or exception.

2 (c) 3

▼ One of the advantages of linked list is the ability to insert data into the list easily. Explain with your own words and figures how to insert data at the beginning, after a given node, at the end and to a sorted list

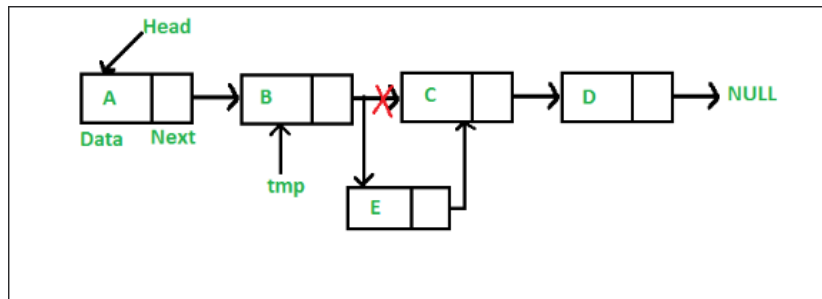
Inserting at the Beginning:

To insert data at the beginning of a linked list, we need to create a new node with the desired data and update the pointers accordingly. The new node becomes the new head of the linked list.



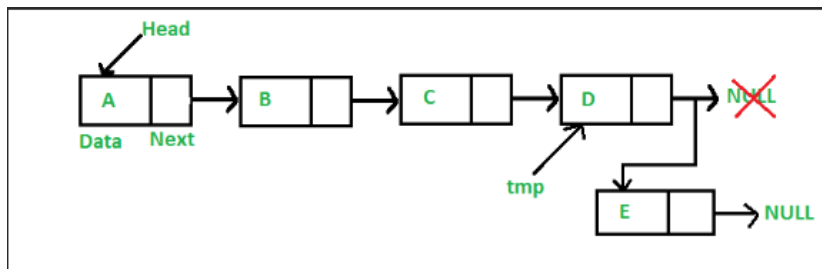
Inserting After a Given Node:

To insert data after a given node in a linked list, we need to create a new node with the desired data and update the pointers of the existing nodes accordingly.



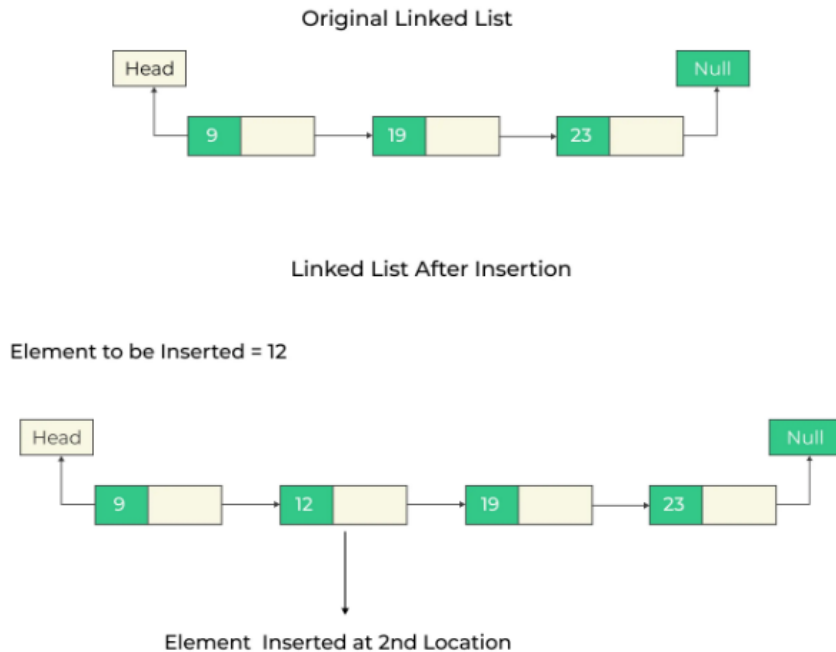
Inserting at the End:

To insert data at the end of a linked list, we need to create a new node with the desired data and update the pointers accordingly. The new node becomes the new last node of the linked list.



Inserting to a Sorted List:

To insert data into a sorted linked list, we need to find the appropriate position where the data should be inserted while maintaining the sorted order. Then, create a new node with the desired data and update the pointers accordingly.



3 (a) 3

- ▼ Define stack. Explain the usage of stack in recursive algorithm implementation

Definition of a Stack:

Stack is a linear data structure that follows Last In First out (LIFO) principle, in which elements can be inserted and removed from one end known as the top of the stack. The operations performed on a stack are often referred to as push (to insert an element) and pop (to remove the top element).

Usage of Stack in Recursive Algorithm Implementation:

Stacks are extensively used in the implementation of recursive algorithms. A recursive algorithm is an algorithm that calls itself to solve a smaller instance of the same problem. During recursion, the computer keeps track of the state of each recursive call by using a stack, known as the call stack or execution stack.

Here's how a stack is used in recursive algorithm implementation:

1. Pushing onto the Stack:

When a recursive function is called, its parameters and local variables are stored on the call stack. The function's return address, which determines where the execution should resume after the function call, is also pushed onto the stack. This process continues as the recursive calls progress deeper into the algorithm.

2. Popping from the Stack:

As the recursive calls complete their execution, they are popped off the stack one by one. This means that the program returns to the point immediately after the function call, restoring the previous state.

3. Maintaining State:

The stack keeps track of the sequence of recursive function calls, allowing the algorithm to maintain the necessary state information for each invocation. The state includes variables, return addresses, and other context-specific data.

4. Base Case:

Recursive algorithms typically have a base case, which defines the termination condition for the recursion.

When the base case is reached, the recursive calls stop, and the algorithm starts unwinding the call stack, popping each frame and returning the results back to the previous invocation.

3 (b) 3.25

- ▼ Simulate the infix to postfix transformation algorithm for Q: $A + (B * C - (D / E \uparrow F) * G) * H$ by showing the stack's contents as each element is scanned #2020 #2014(same but different values)

Input String	Output Stack	Operator Stack
Q:A+(B*C-(D/E↑F)*G)*H	Q	
Q:A+(B*C-(D/E↑F)*G)*H	Q:	
Q:A+(B*C-(D/E↑F)*G)*H	Q:A	
Q:A+(B*C-(D/E↑F)*G)*H	Q:A	+
Q:A+(B*C-(D/E↑F)*G)*H	Q:A	+(
Q:A+(B*C-(D/E↑F)*G)*H	Q:AB	+(
Q:A+(B*C-(D/E↑F)*G)*H	Q:AB	+(*
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC	+(*
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*	+(-
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*	+(-(
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*D	+(-(
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*D	+(-(/
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE	+(-(/
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑	+(-(/
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F	+(-(/
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/	+(-
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/	+(-*
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/G	+(-*
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/G*-	+
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/G*-	++
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/G*-H	++
Q:A+(B*C-(D/E↑F)*G)*H	Q:ABC*DE↑F/G*-H*+	

Infix to Postfix table

Sr No	Expression	Stack	Postfix
0	Q		Q
1	:	:	Q
2	A	:	QA
3	+	:+	QA
4	(	:+(	QA
5	B	:+(	QAB
6	*	:+(*	QAB
7	C	:+(*	QABC
8	-	:+(-	QABC*
9	(	:+(-(	QABC*
10	D	:+(-(	QABC*D
11	/	:+(-(/	QABC*D
12	E	:+(-(/	QABC*DE
13	†	:+(-(†	QABC*DE/
14	F	:+(-(†	QABC*DE/F
15	)	:+(-	QABC*DE/F†
16	*	:+(-*	QABC*DE/F†
17	G	:+(-*	QABC*DE/F†G
18	)	:+	QABC*DE/F†G*-
19	*	:+*	QABC*DE/F†G*-
20	H	:+*	QABC*DE/F†G*-H
21			QABC*DE/F†G*-H*+:

3 (c) 2.5

▼ Write down a routine to insert an element onto a queue

routine to insert an element onto a queue:

```
enqueue(element):  
    1. Create a new node with the given element.  
    2. If the queue is empty, set both the front and rear pointers to the new node.  
    3. Otherwise, set the next pointer of the current rear node to the new node.  
    4. Update the rear pointer to the new node.
```

Explanation:

1. Create a new node with the given element. This node will store the new element.
2. If the queue is empty, meaning there are no existing nodes, set both the front and rear pointers of the queue to the new node. This indicates that the new node is both the first and last node in the queue.
3. If the queue is not empty, set the next pointer of the current rear node (the node currently at the end of the queue) to point to the new node. This links the new node to the existing queue.
4. Update the rear pointer to point to the new node. This ensures that the new node becomes the new rear node of the queue.

4 (a) 2

▼ What is a hash function?

A hash function is a mathematical function that takes an input (or key) and returns a fixed-size string of characters, which is typically a hash value or hash code. The primary purpose of a hash function is to efficiently map data of arbitrary size to a fixed-size value.

4 (b) 3

▼ The keys 14, 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and plus 3 probing. What is the resultant hash table?

To determine the resultant hash table, we will follow the steps of inserting the given keys (14, 12, 18, 13, 2, 3, 23, 5, and 15) into an initially empty hash table of length 10 using open addressing with the hash function $h(k) = k \bmod 10$ and plus 3 probing.

1. Initialize an empty hash table of length 10.
2. Insert the keys one by one, applying the hash function and handling collisions using plus 3 probing:
 - Insert key 14:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(14) = 14 \bmod 10 = 4$.
 - Check if the slot at index 4 is empty. Since it is empty, insert 14 in that slot.

Index	Key
4	14

- Insert key 12:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(12) = 12 \bmod 10 = 2$.
 - Check if the slot at index 2 is empty. Since it is empty, insert 12 in that slot.

Index	Key
4	14
2	12

- Insert key 18:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(18) = 18 \bmod 10 = 8$.
 - Check if the slot at index 8 is empty. Since it is empty, insert 18 in that slot.

Index	Key
4	14
2	12
8	18

- Insert key 13:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(13) = 13 \bmod 10 = 3$.
 - Check if the slot at index 3 is empty. Since it is empty, insert 13 in that slot.

Index	Key
4	14
2	12
8	18
3	13

- Insert key 2:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(2) = 2 \bmod 10 = 2$.
 - A collision occurs at index 2. Apply plus 3 probing: $2 + 3 = 5$.
 - Check if the slot at index 5 is empty. Since it is empty, insert 2 in that slot.

Index	Key
4	14
2	12
8	18
3	13
5	2

- Insert key 3:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(3) = 3 \bmod 10 = 3$.
 - A collision occurs at index 3. Apply plus 3 probing: $3 + 3 = 6$.
 - Check if the slot at index 6 is empty. Since it is empty, insert 3 in that slot.

Index	Key
4	14
2	12
8	18
3	13

5	2
6	3

- Insert key 23:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(23) = 23 \bmod 10 = 3$.
 - A collision occurs at index 3. Apply plus 3 probing: $3 + 3 = 6$.
 - A collision occurs at index 6. Apply plus 3 probing: $6 + 3 = 9$.
 - Check if the slot at index 9 is empty. Since it is empty, insert 23 in that slot.

Index	Key
4	14
2	12
8	18
3	13
5	2
6	3
9	23

- Insert key 5:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(5) = 5 \bmod 10 = 5$.
 - A collision occurs at index 5. Apply plus 3 probing: $5 + 3 = 8$.
 - A collision occurs at index 8. Apply plus 3 probing: $8 + 3 = 1$.
 - Check if the slot at index 1 is empty. Since it is empty, insert 5 in that slot.

Index	Key
4	14
2	12
8	18
3	13
5	2
6	3
9	23
1	5

- Insert key 15:
 - Calculate the hash value using $h(k) = k \bmod 10$: $h(15) = 15 \bmod 10 = 5$.
 - A collision occurs at index 5. Apply plus 3 probing: $5 + 3 = 8$.
 - A collision occurs at index 8. Apply plus 3 probing: $8 + 3 = 1$.
 - A collision occurs at index 1. Apply plus 3 probing: $1 + 3 = 4$.
 - Check if the slot at index 4 is empty. Since it is empty, insert 15 in that slot.

Index	Key
4	14

2	12
8	18
3	13
5	2
6	3
9	23
1	5
7	15

The resultant hash table after inserting all the keys using open addressing with hash function $h(k) = k \bmod 10$ and plus 3 probing is as follows:

| Index | Key |

	-----		-----	
	4		14	
	2		12	
	8		18	
	3		13	
	5		2	
	6		3	
	9		23	
	1		5	
	7		15	
	-----		-----	

4 (c) 3.75

A hash table of length 10 uses open addressing with hash function $h(k)=k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below.

		42	23	34	52	46	33		
0	1	2	3	4	5	6	7	8	9

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- (A) 46,42,34,52,23,33
- (B) 34,42,23,52,33,46
- (C) 46,34,42,23,52,33
- (D) 42,46,33,23,34,52

▼ Answer

Correct Answer - Option 3 : 46, 34, 42, 23, 52, 33

The correct answer is **option 3**.

Concept:

Linear Probing:

We linearly prob for the next slot.

n is the no. of slot

if slot $hash(k) \% n$ is full, then we go to the next slot to search $(hash(k) + 1) \% n$

if slot $(hash(k) + 1) \% n$ is full, then we go to again next slot to search $(hash(k) + 2) \% n$

if again full then we go to check the next index slot **otherwise** insert that element into that index slot.

Requirement:

Inserting 6 values in the empty hash table such as order must be the same as given in the hash table

means inserting every value that **matches the index** with the given hash table

Then **fulfill the requirement** otherwise **fails the requirement**.

Option 1: Not possible

Inserting these values 46, 42, 34, 52, 23, 33

Apply the procedure as given above for inserting the value in the hash table

$h(46) = 46 \% 10 = 6 \rightarrow$ match the index

$h(42) = 42 \% 10 = 2 \rightarrow$ match the index

$h(34) = 34 \% 10 = 4 \rightarrow$ match the index

$h(52) = 52 \% 10 = 2 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 1) \% 10 = 3 \rightarrow$ Not match the index

Option 2: Not possible

Inserting these values 34, 42, 23, 52, 33, 46

$h(34) = 34 \% 10 = 4 \rightarrow$ match the index

$h(42) = 42 \% 10 = 2 \rightarrow$ match the index

$h(23) = 23 \% 10 = 3 \rightarrow$ match the index

$h(52) = 52 \% 10 = 2 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 1) \% 10 = 3 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 2) \% 10 = 4 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 3) \% 10 = 5 \rightarrow$ match the index

$h(33) = 33 \% 10 = 3 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 1) \% 10 = 4 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 2) \% 10 = 5 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 3) \% 10 = 6 \rightarrow$ Not match the index

Option 3: Possible

Inserting these values

46, 34, 42, 23, 52, 33

$h(46) = 46 \% 10 = 6 \rightarrow$ match the index

$h(34) = 34 \% 10 = 4 \rightarrow$ match the index

$h(42) = 42 \% 10 = 2 \rightarrow$ match the index

$h(23) = 23 \% 10 = 3 \rightarrow$ match the index

$h(52) = 52 \% 10 = 2 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 1) \% 10 = 3 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 2) \% 10 = 4 \rightarrow$ conflict with the same index in this table

$h(52) = (52 + 3) \% 10 = 5 \rightarrow$ match the index

$h(33) = (33 + 1) \% 10 = 4 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 2) \% 10 = 5 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 3) \% 10 = 6 \rightarrow$ conflict with the same index in this table

$h(33) = (33 + 4) \% 10 = 7 \rightarrow$ match the index

In this option, all the value **matches with the same index** with the given hash table

So, this fulfills the requirement

Option 4: Not possible

Inserting these values 42, 46, 33, 23, 34, 52

Apply the procedure as given above for inserting the value in the hash table

$h(42) = 42 \% 10 = 2 \rightarrow$ match the index

$h(46) = 46 \% 10 = 6 \rightarrow$ match the index

$h(33) = 33 \% 10 = 3 \rightarrow$ Not match the index

Hence not possible

5 (a) 3

- ▼ Define the terms (i) siblings, (ii) ancestor, and (iii) depth of a binary tree **#revised-1**

(i) Siblings

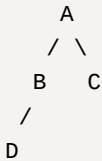
Siblings in a binary tree are nodes that have the same parent node. For example,



nodes B and C are siblings because they both have node A as their parent node

Ancestor:

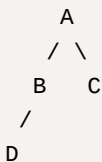
In a binary tree, a node is an ancestor of another node if it is on the path from the root node to that node. For example,



node A is an ancestor of nodes B, C, and D

Depth of a node

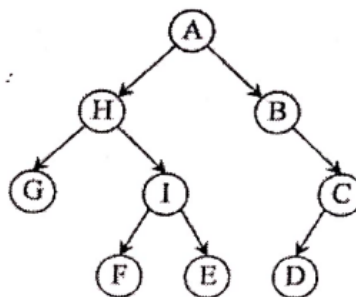
The depth of a node in a binary tree is the number of edges on the path from the root node to that node. For example,



the depth of node D is 2

5 (b) 5.75

Tree traversal (also known as walking the tree) refers to the process of visiting each node exactly once. Simulate the preorder traversal algorithm for the following tree **#revised-1**



▼ Answer

Preorder Traversal Algorithm:

1. Visit the current node.
2. Recursively traverse the left subtree.
3. Recursively traverse the right subtree.

Applying the algorithm to the given tree, the preorder traversal sequence will be as follows:

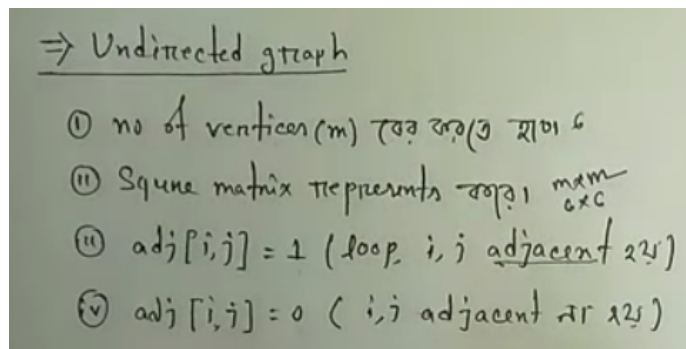
1. Start at the root node A.
 2. Visit node A.
 3. Move to the left subtree and visit node H.
 4. Move to the left subtree of H and visit node G.
 5. Move back up to node H and visit its right subtree. Visit node I.
 6. Move to the left subtree of I and visit node F.
 7. Move back up to node I and visit its right subtree. Visit node E.
 8. Move back up to node I and move back up to node H.
 9. Move back up to the root node A and move to its right subtree.
 10. Visit node B.
 11. Move to the right subtree of B and visit node C.
 12. Move to the left subtree of C and visit node D.
 13. Move back up to node C and move back up to node B.
 14. Move back up to the root node A.
- The preorder traversal sequence for the given tree is: A, H, G, I, F, E, B, C, D.

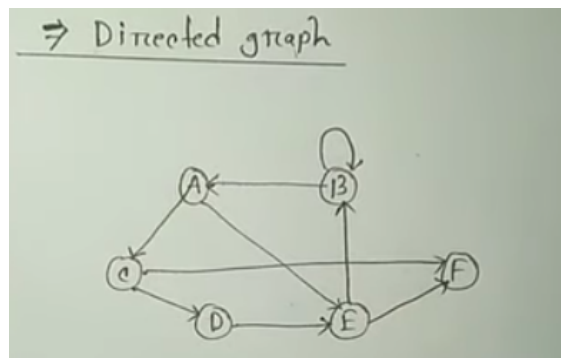
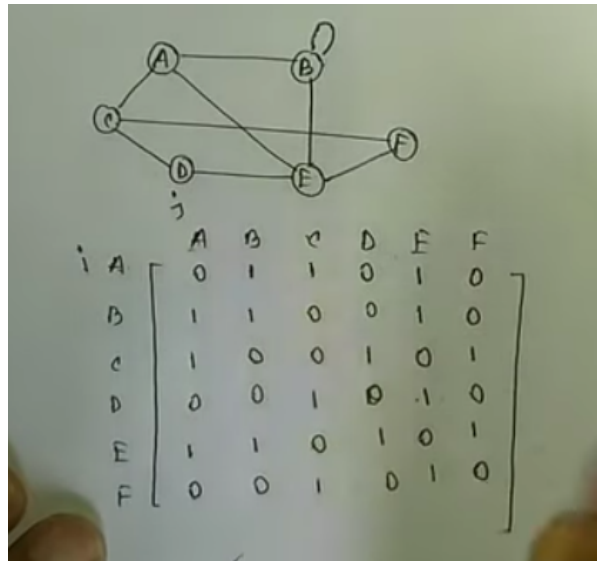
6 (a) 2.75

▼ What is adjacency matrix? How is it formed? **#revised-1**

An adjacency matrix is a square matrix used to represent a graph. The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph.

To form an adjacency matrix for a graph, we create a square matrix with the same number of rows and columns as the number of vertices in the graph. The element at row i , column j of the matrix is 1 if there is an edge from vertex i to vertex j , and 0 otherwise.





	A	B	C	D	E	F
A	0	0	1	0	1	0
B	1	1	0	0	1	0
C	0	0	0	1	0	1
D	0	0	0	0	1	0
E	0	1	0	0	0	0
F	0	0	0	0	0	0

Adjacency Matrix | Graph Representation | Data Structure | Bangla Tutorial

In this video i have discussed about the topic of Adjacency Matrix of graph representation in data structure.

https://www.youtube.com/watch?v=9QTHtIAVhmU&ab_channel=FarhanHossan



Md. Farhan Hossain
B.Sc. in CSE

Adjacency Matrix Graph Representation

Data Structure & Algorithm

6 (b) 3

▼ Define the terms (i) Isolated node, (ii) Simple Path and (iii) Weighted Graph #revised-1

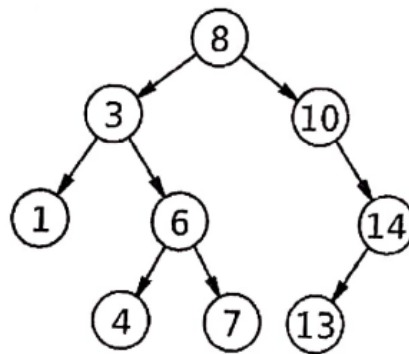
1. Isolated node: An isolated node, also known as an isolated vertex, is a node in a graph that is not connected to any other nodes. In other words, it does not have any edges or connections to other nodes in the graph.

Isolated nodes are independent and do not participate in the connectivity or structure of the graph.

2. Simple Path: In a graph, a simple path refers to a sequence of vertices where each pair of consecutive vertices in the sequence is connected by an edge. The path does not contain any repeated vertices, meaning that each vertex is visited only once. Additionally, a simple path does not have any loops or cycles, ensuring that there are no repeated edges in the path.
3. Weighted Graph: A weighted graph is a type of graph in which each edge is assigned a numerical value called a weight. The weight represents some quantitative measure, such as the cost, distance, or capacity associated with that edge. In a weighted graph, the edges carry additional information beyond the connectivity between vertices.

6 (c) 3

How can you search '5' in the following binary search tree? However if '5' is not in the tree just insert to its appropriate place and show the resultant tree. **#revised-1**



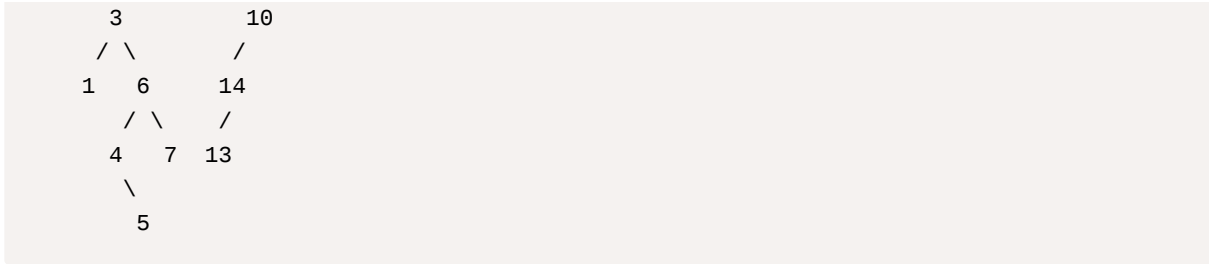
▼ Answer

To search for the value '5' in the given binary search tree and insert it if it is not present, follow these steps:

1. Start at the root node (value 8).
2. Compare the value '5' with the current node's value (8).
 - Since '5' is less than 8, move to the left child.
3. Compare the value '5' with the current node's value (3).
 - Since '5' is greater than 3, move to the right child.
4. Compare the value '5' with the current node's value (6).
 - Since '5' is less than 6, move to the left child.
5. Compare the value '5' with the current node's value (4).
 - Since '5' is greater than 4, move to the right child.
6. The right child of the current node is null, indicating an appropriate place to insert '5'.
 - Insert '5' as the right child of the current node (4).

The resultant tree after searching '5' and inserting it at the appropriate place is:





7 (a) 2

▼ What is meant strongly and weakly connected in a graph? #revised-1

a strongly connected graph is a graph in which every vertex is reachable from every other vertex, regardless of the direction of the edges. A weakly connected graph, on the other hand, is a graph in which every vertex is reachable from every other vertex, but only if we ignore the direction of the edges.

For example,

Strongly connected graph:

`A -> B -> C -> A`

In this graph, there is a path from every vertex to every other vertex.

Weakly connected graph:

`A -> B -> C`
`D -> E -> F`

For example, there is a path from A to F, even though there is no direct edge from A to F.

Strongly Connected Graph:

A directed graph is considered strongly connected if there is a directed path between every pair of vertices in the graph. In other words, for any two vertices u and v in the graph, there exists a directed path from u to v and also from v to u .

Weakly Connected Graph:

A directed graph is considered weakly connected if replacing all the directed edges with undirected edges results in a connected undirected graph. In other words, if we ignore the direction of edges and treat them as undirected, the resulting graph is connected.

In graph theory, connectivity is a fundamental concept that describes how well the vertices (nodes) in a graph are connected to each other. Two important types of connectivity in a graph are "strong connectivity" and "weak connectivity."

1. Strong Connectivity:

A directed graph is said to be strongly connected if there is a directed path between every pair of vertices in the graph. In other words, for every two vertices u and v in a strongly connected graph, there is a directed path from u to v , and there is also a directed path from v to u . This means that we can reach any vertex in the graph from any other vertex by following directed edges.

To illustrate strong connectivity, consider the following directed graph:

cssCopy code

```

A ---> B ---> C
^       |
|       v
  
```

```
D <---
```

In this graph, we can find directed paths between every pair of vertices:

- A to B: $A \rightarrow B$
- B to C: $B \rightarrow C$
- C to A: $C \rightarrow A$
- A to D: $A \rightarrow B \rightarrow D$
- D to B: $D \rightarrow B$
- B to D: $B \rightarrow D$
- D to A: $D \rightarrow B \rightarrow A$
- C to D: $C \rightarrow A \rightarrow B \rightarrow D$

Since there is a directed path between every pair of vertices, this graph is strongly connected.

1. Weak Connectivity:

A directed graph is said to be weakly connected if it becomes connected when we ignore the directions of the edges and treat them as undirected edges. In other words, if we disregard the directions of the edges, the underlying undirected graph is connected. It means that we can reach any vertex from any other vertex by traversing either directed or undirected edges.

To illustrate weak connectivity, consider the following directed graph:

```
cssCopy code
A ---> B ---> C
^       |
|       v
D <---
```

If we ignore the directions of the edges, the undirected version of this graph looks like this:

```
cssCopy code
A --- B --- C
|         |
|         |
D -----
```

In this undirected graph, we can find undirected paths (ignoring edge directions) between every pair of vertices:

- A to B: $A - B$
- B to C: $B - C$
- C to A: $C - A$
- A to D: $A - D$
- D to B: $D - B$

- B to D: B - D
- D to A: D - A
- C to D: C - A - D

Since there is a path between every pair of vertices in the undirected graph, the original directed graph is weakly connected.

In summary:

- Strong connectivity in a directed graph requires a directed path between every pair of vertices.
- Weak connectivity in a directed graph considers the underlying undirected graph and requires a path (ignoring edge directions) between every pair of vertices.

Both strong and weak connectivity concepts are important in graph theory and have various applications in areas like network analysis, transportation, and social networks.

7 (b) 2.75

- ▼ Prove that the maximum number of edges that a graph with n vertices is $n*(n-1)/2$ #revised-1

For Simple Graph:

7. Maximum edges in Simple Graph is $n(n-1)/2$ || Number of edges in Complete Graph is $n(n-1)/2$ ||

Theorem Statement:

1. The maximum number of edges in a simple graph with n vertices is $n(n-1)/2$
2. The number of edges in the complete graph is $n(n-1)/2$.

📺 <https://www.youtube.com/watch?v=l8jo1z8BAGU>

GRAPH THEORY #7

Theorem on Graph Th

- The maximum number of edges in a simple graph with n vertices is $n(n-1)/2$.
- The number of edges in a complete graph is $n(n-1)/2$.

proof: Since the graph is simple. It has no self loops & parallel edges. Since it has n vertices. Each vertex is adjacent to almost $(n-1)$ vertices

ie. the degree of each vertex is almost $(n-1)$.
 $\therefore$ Sum of degree of all vertices is $n(n-1)$
 we know that;
 $\therefore$ Sum of degree of all vertex = $2 \times$ no. of edges

$$n(n-1) = 2 \cdot \text{number of edges}$$

$$\text{number of edges} = \frac{n(n-1)}{2}$$

Thus, the maximum no. of edges in simple graph with n -vertices is $\frac{n(n-1)}{2}$.

Considering a simple graph, that has no self loops and parallel edges. Since it has n vertices. Each vertex is adjacent to at most $(n-1)$ vertices.

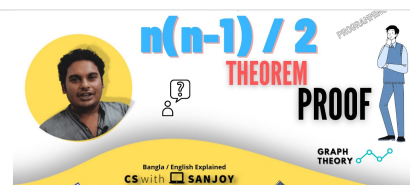
i.e the degree of each vertex is at most $(n-1)$
 $\therefore$ Sum of degree of all vertices is $n(n-1)$.
We know that,
Sum of degree of all vertex = $2 \times$ no. of edges.
 $\Rightarrow n(n-1) = 2 \times$ no. of edges
 $\Rightarrow$ no. of edges = $\frac{n(n-1)}{2}$
Thus, the maximum no. of edges a graph with n vertices is $\frac{n(n-1)}{2}$

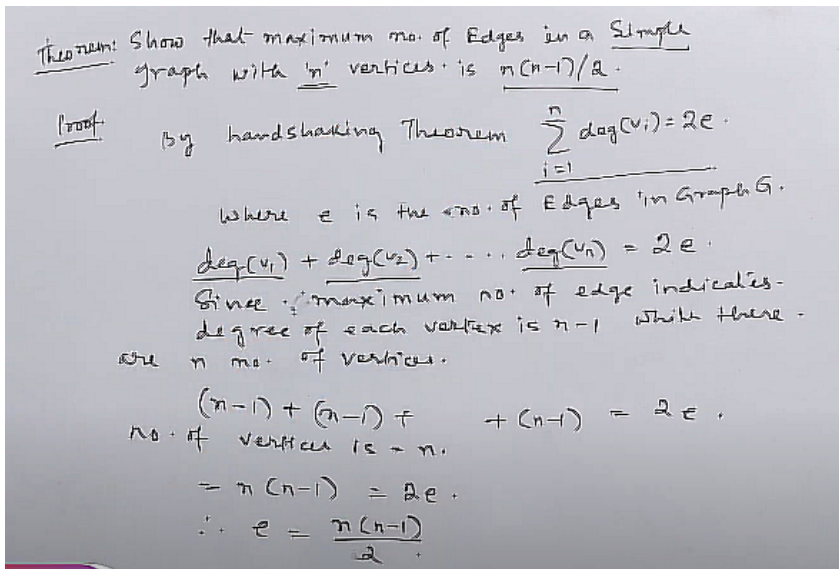
For Complete Graph:

Complete graph | $n(n-1)/2$

Topic - A complete graph with n vertices has maximum $n(n-1)/2$ edges.

 <https://www.youtube.com/watch?v=9Vvd6NDnCZI>





A complete graph is a simple undirected graph in which each pair of distinct vertices is connected by an edge. In other words, there is an edge between every pair of vertices in a complete graph.

Let's assume we have a graph with n vertices, and we want to find the maximum number of edges it can have.

In a complete graph, each vertex is connected to every other vertex with an edge. So, the first vertex has $(n-1)$ edges, the second vertex has $(n-1)$ edges, the third vertex has $(n-1)$ edges, and so on, until the last vertex (n -th vertex) has $(n-1)$ edges.

To find the total number of edges in the complete graph, we can sum up the number of edges for each vertex:

Total number of edges = $(n-1) + (n-1) + (n-1) + \dots + (n-1)$

Since there are n vertices in the graph, the above summation can be written as:

Total number of edges = $n * (n-1)$

However, in a complete graph, each edge is counted twice (since it connects two vertices). For example, the edge between vertex A and vertex B is the same as the edge between vertex B and vertex A.

So, the actual number of distinct edges in the complete graph is half of the total number of edges calculated above:

Number of distinct edges = $n * (n-1) / 2$

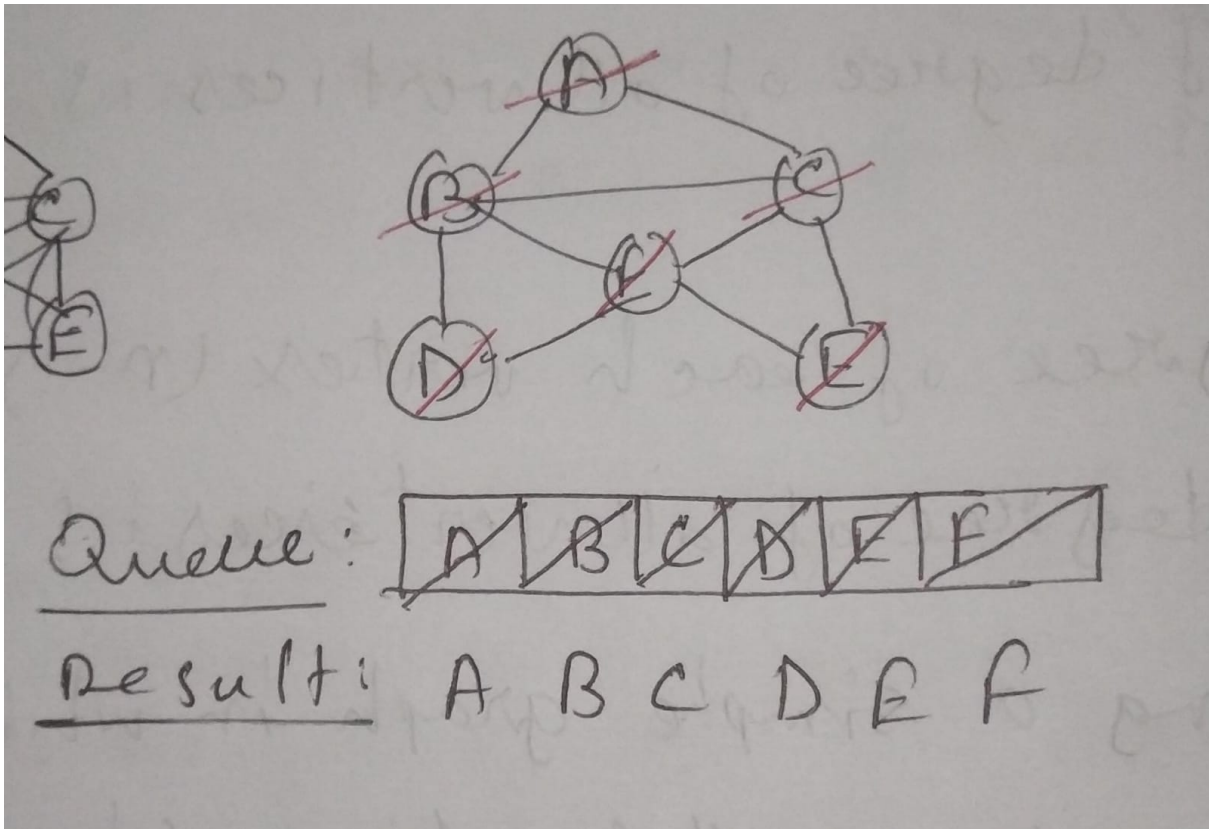
Therefore, the maximum number of edges that a graph with n vertices can have is $n * (n-1) / 2$.

7 (c) 4

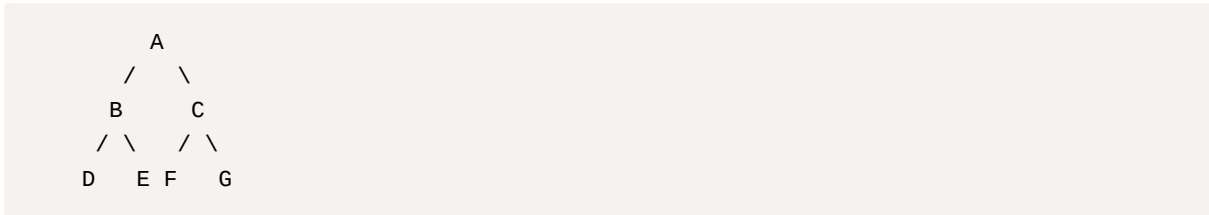
▼ Explain Breadth First search algorithm with example #revised-1

It is an algorithm that is used for traversing structures. The algorithm efficiently visits and marks all the key nodes in a graph, this algorithm selects a single node in a graph and then visits all the adjacent nodes of the selected node.

Once the algorithm visits and marks the starting node, then it moves towards the nearest unvisited nodes and analyses them. Once visited, all nodes are marked. These iterations continue until all the nodes of the graph have been successfully visited and marked. For example,



For example considering the following undirected graph:



Step 1: Let's start from vertex A.

Step 2: Enqueue vertex A into a queue and mark it as visited.

Queue: [A]

Visited: {A}

Step 3: Dequeue the front vertex (A) from the queue and explore its neighbors.

- Vertex A has two neighbors: B and C. Enqueue them into the queue (if not already visited) and mark them as visited.

Queue: [B, C]

Visited: {A, B, C}

Step 4: Dequeue the front vertex (B) from the queue and explore its neighbors.

- Vertex B has two neighbors: D and E. Enqueue them into the queue (if not already visited) and mark them as visited.

Queue: [C, D, E]

Visited: {A, B, C, D, E}

Step 5: Dequeue the front vertex (C) from the queue and explore its neighbors.

- Vertex C has two neighbors: F and G. Enqueue them into the queue (if not already visited) and mark them as visited.

Queue: [D, E, F, G]

Visited: {A, B, C, D, E, F, G}

Step 6: Dequeue the front vertex (D) from the queue. As D has no unvisited neighbors, move to the next vertex in the queue.

Queue: [E, F, G]

Visited: {A, B, C, D, E, F, G}

Step 7: Dequeue the front vertex (E) from the queue. As E has no unvisited neighbors, move to the next vertex in the queue.

Queue: [F, G]

Visited: {A, B, C, D, E, F, G}

Step 8: Dequeue the front vertex (F) from the queue. As F has no unvisited neighbors, move to the next vertex in the queue.

Queue: [G]

Visited: {A, B, C, D, E, F, G}

Step 9: Dequeue the front vertex (G) from the queue. As G has no unvisited neighbors, move to the next vertex in the queue.

Queue: []

Visited: {A, B, C, D, E, F, G}

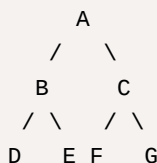
Step 10: The queue is empty, and all vertices are visited. The BFS traversal is complete.

Breadth-First Search (BFS) is a graph traversal algorithm that explores the vertices of a graph level by level. It starts from a specified source vertex and visits all its neighbors before moving on to their neighbors. BFS uses a queue data structure to keep track of the vertices to be visited, ensuring that vertices are visited in the order they were discovered.

Here's a step-by-step explanation of the Breadth-First Search algorithm with an example:

Example Graph:

Consider the following undirected graph:



Explanation of BFS:

1. Choose a starting vertex as the source. Let's start with vertex A.
2. Mark A as visited and enqueue it into the BFS queue. The queue now contains [A].
3. Dequeue the first vertex from the queue (A), and explore its neighbors.
 - Visit B and mark it as visited. Enqueue B into the queue. The queue now contains [B].
 - Visit C and mark it as visited. Enqueue C into the queue. The queue now contains [B, C].

4. Dequeue the first vertex from the queue (B), and explore its neighbors.
 - Visit D and mark it as visited. Enqueue D into the queue. The queue now contains [C, D].
 - Visit E and mark it as visited. Enqueue E into the queue. The queue now contains [C, D, E].
5. Dequeue the first vertex from the queue (C), and explore its neighbors.
 - Visit F and mark it as visited. Enqueue F into the queue. The queue now contains [D, E, F].
 - Visit G and mark it as visited. Enqueue G into the queue. The queue now contains [D, E, F, G].
6. Dequeue the first vertex from the queue (D), and explore its neighbors.
 - Since D has no unvisited neighbors, move on to the next vertex in the queue.
7. Dequeue the first vertex from the queue (E), and explore its neighbors.
 - Since E has no unvisited neighbors, move on to the next vertex in the queue.
8. Dequeue the first vertex from the queue (F), and explore its neighbors.
 - Since F has no unvisited neighbors, move on to the next vertex in the queue.
9. Dequeue the first vertex from the queue (G), and explore its neighbors.
 - Since G has no unvisited neighbors, move on to the next vertex in the queue.
10. The queue is empty, and all vertices have been visited. The BFS traversal is complete.

The BFS algorithm visits the vertices in the following order: A, B, C, D, E, F, G. This order represents a level-by-level exploration of the graph starting from vertex A. The BFS algorithm guarantees that all vertices reachable from the source vertex will be visited, and it is useful for finding the shortest path in an unweighted graph.

8 (a) 2

- ▼ Suppose we want to encode a message constructed from the symbols A, B, C, D, E, F and G using a fixed-length code. How many bits are required to encode the message FDEGAACAAGAAFABA?

8(a)

Total = 16 characters

Character	Count	Code
A	8	000
B	1	001
C	1	010
D	1	011
E	1	100
F	2	101
G	2	110

no. of Characters

$= 7$

let, no. of bits = n

then $2^n = 2^2 = 4$

$2^3 = 8$

∴ take 3 bits

to represent 1 character

Character in

ASCII = $7 \times 8 = 56$ bits

Character code = $7 \times 3 = 21$ bits

∴ total bits for table = $56 + 21 = 77$ bits

The string = $16 \times 3 = 48$ bits

∴ Total message size = $77 + 48$

= 125 bits

8 (b) 3.75

▼ Build the Huffman coding tree for the message 'science engineering'

science engineering

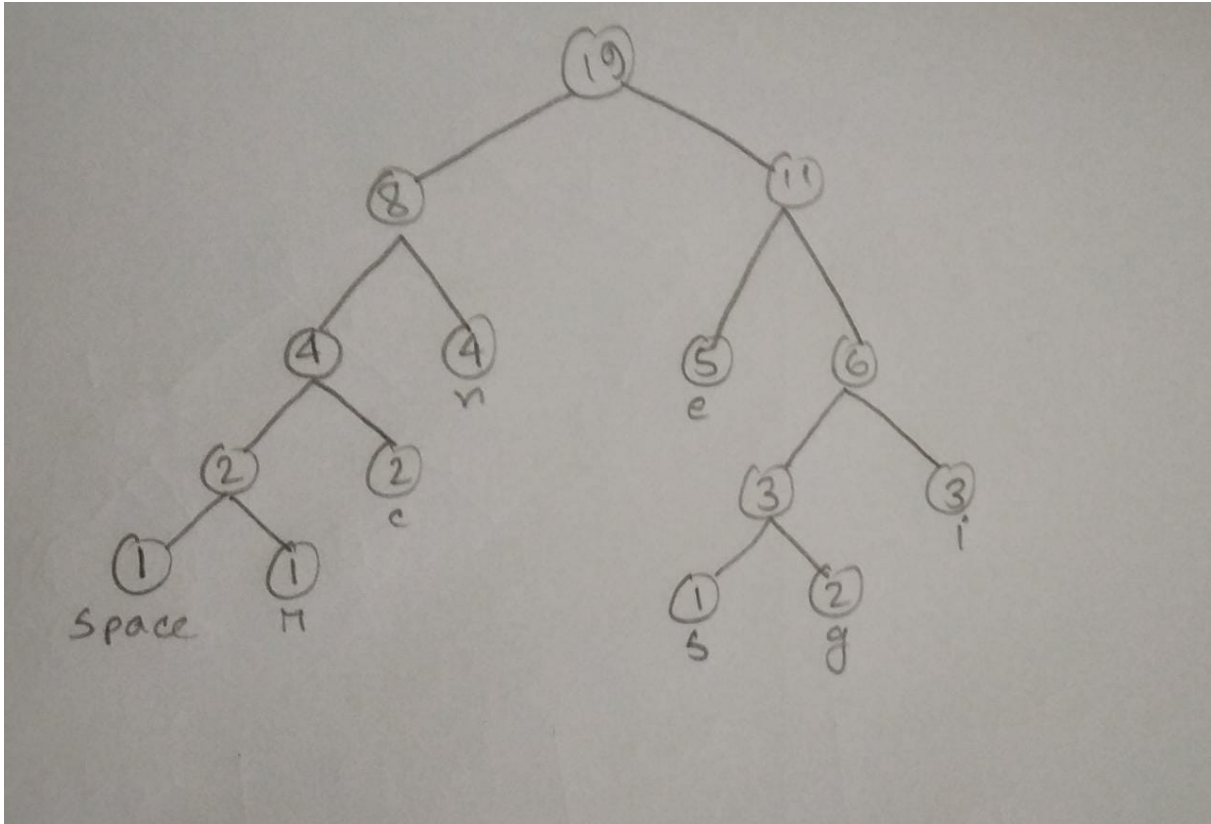
~~s=1 c=2~~,
~~c=2~~ ; ~~e=5~~, ~~s=1~~, ~~i=3~~, ~~n=4~~, ~~n=1~~, ~~g=2~~

e=5,
 n=4,
 i=3,
 c=2,
 g=2,
 s=1,
 n=1,
~~g=2~~

space = 1

(10) A = AX1

abc x twice	abc	twice
8 = 1x8	1	8
1100	1	1
0100	1	1



8 (c) 3

Suppose you are to insert a node 'X' as the right child of node 'P'. Discuss the inserting mechanisms with figures.

#revised-1

- When the right subtree of 'P' is empty
- When the right subtree of 'P' is not empty

▼ Answer

- When the right subtree of 'P' is empty:

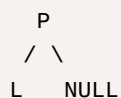
In this scenario, 'P' does not have any right child, so we simply create a new node 'X' and make it the right child of 'P'.

Step 1: Create node 'X' with its data value.

Step 2: Set the right pointer of 'P' to point to 'X'.

Assuming node 'P' has a left child 'L'

Before Insertion:



After Insertion:



2. When the right subtree of 'P' is not empty:

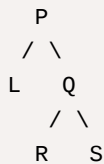
In this scenario, 'P' already has a right child, so we need to insert 'X' in the right subtree of 'P' following the appropriate rule of a binary search tree.

Assuming node 'Q' is the right child of node 'P' and node 'R' and 'S' are the children of node 'Q'

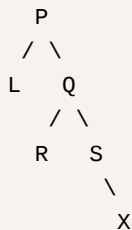
Step 1: Traverse the right subtree of 'P' starting from 'Q' and following the appropriate rule (e.g., if 'P' is in a binary search tree, insert 'X' to the right of 'Q' if 'X' is greater than 'Q', or if 'P' is in a general binary tree, find the appropriate position to insert 'X').

Step 2: Once we find the correct position, create node 'X' and insert it at the appropriate place in the right subtree of 'P'.

Before Insertion:



After Insertion:



2020

1 (a) 2

▼ How can you differ linear data structures from non-linear data structures? **#revised-1** #2015

Linear Data Structures	Non-Linear Data Structures
A data structure is linear when all the elements are arranged in a linear or sequential order	A data structure is non linear when all the elements are not arranged in a linear or sequential order
Examples: Arrays, Linked Lists, Stacks, Queues	Examples: Trees, Graphs
Traversal is straightforward and follows a linear order.	Traversal can be complex and may involve recursive methods.
Access time for elements is usually constant or $O(1)$ in most cases.	Access time may vary and can be $O(\log n)$ or $O(n)$ depending on the specific structure.
Each element in a linear data structure has a unique predecessor and successor, except for the first and last elements	Each element in a non-linear data structure can have multiple predecessors and successors

Linear Data Structures:

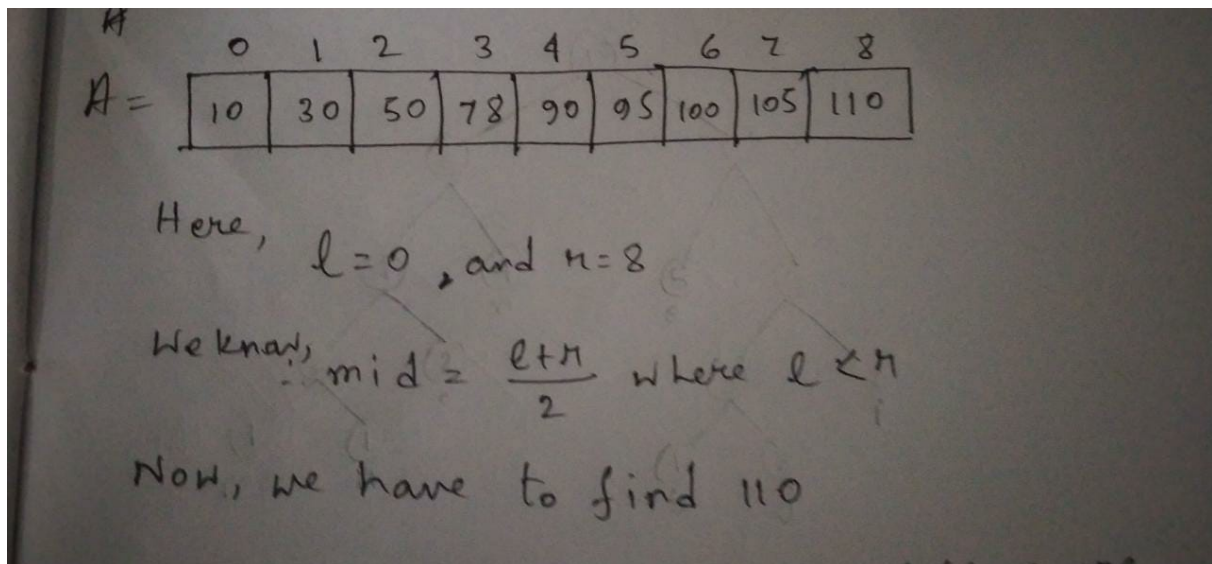
1. Linear data structures organize and store data elements in a sequential manner, where each element is connected to its previous and next element (except the first and last elements).
2. They are typically represented as a sequence or list of elements.
3. The elements in linear data structures can be accessed in a linear fashion, one after the other, starting from the first element to the last.
4. Examples of linear data structures include arrays, linked lists, stacks, and queues.

Non-Linear Data Structures:

1. Non-linear data structures do not store elements in a sequential manner, and the elements may be interconnected in various ways.
2. They often represent relationships between elements rather than a linear sequence.
3. The elements in non-linear data structures may not be accessed in a linear order, and traversal methods vary based on the structure.
4. Examples of non-linear data structures include trees (binary trees, AVL trees, etc.), graphs (directed and undirected graphs), and heaps.

1 (b) 4

▼ Simulate the binary search algorithm on the following data: 10 30 50 78 90 95 100 105 110 (suppose we search for item 110)



l	h	mid
0	8	4
5	8	6
7	8	7
8	8	8

We found that 110 is at index 8. We can simulate binary search algorithm for the given sorted data by the following steps:

1. Given sorted data: 10 30 50 78 90 95 100 105 110

2. Initialize variables:

- $l = 0$ (the index of the first element)
- $r = 8$ (the index of the last element)
- $data = 110$ (the item we want to find)

3. Start the binary search:

- Iteration 1:
 - $mid = (l + r) / 2 = (0 + 8) / 2 = 4$
 - $data > A[mid] = 110 > 90$
 - Set $l = 4 + 1 = 5$ (the index of the middle element + 1)
- Iteration 2:
 - $mid = (l + r) / 2 = (5 + 8) / 2 = 6$
 - $data > A[mid] = 110 > 100$
 - Set $l = 6 + 1 = 7$ (the index of the middle element + 1)
- Iteration 3:
 - $mid = (l + r) / 2 = (7 + 8) / 2 = 7$
 - $data > A[mid] = 110 > 105$
 - Set $l = 7 + 1 = 8$ (the index of the middle element + 1)
- Iteration 4:
 - $mid = (l + r) / 2 = (8 + 8) / 2 = 8$
 - $data == A[mid] = 110 == 110$
 - The target (110) is found at index 8.

4. The binary search algorithm terminates, and it returns the index 8 as the position of the target element (110) in the sorted data.

Given data: 10, 30, 50, 78, 90, 95, 100, 105, 110

Target item: 110

Step 1:

The data is already sorted in ascending order. We have the following array: [10, 30, 50, 78, 90, 95, 100, 105, 110]

Let's set the search range:

- Start index: 0
- End index: 8 (total elements - 1)

Step 2:

Calculate the middle index of the current search range:

- Middle index = (Start index + End index) / 2 = (0 + 8) / 2 = 4

Step 3:

Compare the middle element (90) with the target item (110):

- $90 < 110$
- Since the middle element is smaller than the target, we update the start index to the next position after the middle index: Start index = Middle index + 1 = 4 + 1 = 5

Step 4:

Now, we have a new search range:

- Start index: 5
- End index: 8

Step 5:

Calculate the new middle index of the current search range:

- Middle index = (Start index + End index) / 2 = (5 + 8) / 2 = 6

Step 6:

Compare the middle element (100) with the target item (110):

- $100 < 110$
- Again, the middle element is smaller than the target, so we update the start index to the next position after the middle index: Start index = Middle index + 1 = 6 + 1 = 7

Step 7:

Now, we have another new search range:

- Start index: 7
- End index: 8

Step 8:

Calculate the new middle index of the current search range:

- Middle index = (Start index + End index) / 2 = (7 + 8) / 2 = 7.5 (Ignore the decimal part)

Step 9:

Compare the middle element (105) with the target item (110):

- $105 < 110$
- The middle element is still smaller than the target, so we update the start index to the next position after the middle index: Start index = Middle index + 1 = 7 + 1 = 8

Step 10:

Now, the start index becomes equal to the end index (8), and we have a single element left to check.

Step 11:

Calculate the new middle index of the current search range:

- Middle index = (Start index + End index) / 2 = (8 + 8) / 2 = 8

Step 12:

Compare the middle element (110) with the target item (110):

- $110 = 110$
- We have found the target element (110) at index 8.

The binary search algorithm successfully found the target element 110 at index 8.

1 (c) 2.75

- ▼ How are the elements of a 2D array stored in the memory? **#revised-1 #2019**

2D arrays are stored in memory in one of two ways: row-major order or column-major order.

- **Row-major order:** In row-major order, the elements of each row are stored contiguously in memory, followed by the elements of the next row, and so on. This is the most common way to store 2D arrays in

memory.

Here is an example of a 2D array stored in row-major order:

```
array = [[1, 2, 3],  
         [4, 5, 6],  
         [7, 8, 9]]
```

In memory, this array would be stored as follows:

```
1, 2, 3, 4, 5, 6, 7, 8, 9
```

- **Column-major order:** In column-major order, the elements of each column are stored contiguously in memory, followed by the elements of the next column, and so on. This is less common than row-major order

Here is an example of a 2D array stored in column-major order:

```
array = [[1, 2, 3],  
         [4, 5, 6],  
         [7, 8, 9]]
```

In memory, this array would be stored as follows:

```
1, 4, 7, 2, 5, 8, 3, 6, 9
```

The elements of each column are stored contiguously in memory, followed by the elements of the next column.

2 (a) 4

- ▼ Score is a 20×5 matrix with base address 200 and $w=4$. Find out the address of the element $\text{Score}[10,2]$ for both row major order and column major order **#revised-1**

Row Major

$$m = 20, n = 5$$
$$B = 200, w = 4$$
$$\therefore \text{Score}[10][2] = 200 + \left[\{(10-1) \times 5\} + (2-1) \right] \times 4$$
$$= 200 + 46 \times 4$$
$$= 384$$

Column Major

$$\begin{aligned}
 \text{Score } [10][2] &= 200 + \{ (2-1) \times 20 \} + (10-1) \times 4 \\
 &= 200 + 116 \\
 &= 316
 \end{aligned}$$

2 (b) 3

▼ What is sparse matrix? What is the difference between triangular matrix and tridiagonal matrix? **#revised-1 #2016**

A **sparse matrix** is a matrix in which most of the elements are zero.

For example,

Sparse Matrix =

```
[ 0 1 0 ]
[ 2 0 3 ]
[ 0 0 4 ]
```

A triangular matrix is a special kind of square matrix. A square matrix is called lower triangular if all the entries above the main diagonal are zero. Similarly, a square matrix is called upper triangular if all the entries below the main diagonal are zero.

For example,

Lower Triangular matrix:

```
[ 1 0 0 ]
[ 2 1 0 ]
[ 3 2 1 ]
```

Upper Triangular matrix:

```
[ 1 2 3 ]
[ 0 1 2 ]
[ 0 0 1 ]
```

A **tridiagonal matrix** is a special type of triangular matrix in which all the elements are zero except for the main diagonal, the diagonal above the main diagonal, and the diagonal below the main diagonal.

For example,

Tridiagonal matrix:

```
[ 1 2 0 ]
[ 2 3 4 ]
[ 0 4 5 ]
```

The difference between a triangular matrix and a tridiagonal matrix is that a triangular matrix can have non-zero elements in any row above or below the main diagonal, depending on whether it is lower or upper triangular matrix. On the other hand, a tridiagonal matrix can only have non-zero elements in the diagonal, the diagonal above the main diagonal, and the diagonal below the main diagonal.

2 (c) 1.75

▼ How can you locate element a_{ij} of a sparse matrix from a 1D array? **#revised-1 #2018 #2017**

We can use the row-major order or column-major order indexing, to locate element a_{ij} of a sparse matrix from a 1D array.

1. Row-Major Order:

Formula: $A[i][j] = B + \{(i * n) + j\} * w$

2. Column-Major Order:

Formula: $A[i][j] = B + \{(j * m) + i\} * w$

Here, B = Base Address

i = Row index

j = Column index

w = Storage size of one element

m = Number of rows given in the matrix.

n = Number of column given in the matrix.

To locate element a_{ij} of a sparse matrix from a 1D array, you can use the row-major order or column-major order indexing. Both methods provide a way to map 2D matrix elements to a 1D array efficiently.

1. Row-Major Order:

In row-major order, the elements of the 2D matrix are stored row by row in the 1D array. To locate element a_{ij} in the 1D array, you can use the following formula:

Index_RowMajor = $(i * \text{number_of_columns}) + j$

Here:

- i is the row index (starting from 0 for the first row).
- j is the column index (starting from 0 for the first column).
- number_of_columns is the total number of columns in the 2D matrix.

Using this formula, you can calculate the index in the 1D array corresponding to element a_{ij} .

1. Column-Major Order:

In column-major order, the elements of the 2D matrix are stored column by column in the 1D array. To locate element

a_{ij} in the 1D array, you can use the following formula:

Index_ColumnMajor = $(j * \text{number_of_rows}) + i$

Here:

- i is the row index (starting from 0 for the first row).
- j is the column index (starting from 0 for the first column).
- number_of_rows is the total number of rows in the 2D matrix.

3 (a) 1.75

- ▼ Mention some differences between arrays and linked lists #2021

3 (b) 2

- ▼ Can binary search algorithm be employed on data stored in linked list? Justify your answer

No, the binary search algorithm cannot be directly employed on data stored in a standard singly linked list. The binary search algorithm requires random access to elements based on their index, which is not directly supported in a linked list.

Justification:

Binary search relies on dividing the sorted data set into two halves and repeatedly discarding one half based on the comparison of the search key with the middle element. This process requires accessing elements at specific positions in the data set, like accessing the middle element or elements at the beginning or end of the sorted list.

However, in a singly linked list, accessing elements based on their index requires traversing the list sequentially from the head (start) or tail (end), which results in linear time complexity, $O(n)$, for each access. Binary search requires logarithmic time complexity, $O(\log n)$, for accessing elements, which cannot be achieved with a singly linked list.

Yes, the binary search algorithm can be employed on data stored in a linked list, but there are some limitations and considerations that need to be taken into account. Let's explore how binary search can be used with a linked list and justify its applicability:

Binary Search Algorithm:

The binary search algorithm is an efficient search algorithm used to find a target element in a sorted array. It works by repeatedly dividing the search space in half until the target element is found or the search space is exhausted. Binary search requires random access to elements, which means it can access elements in constant time using their index.

Binary Search on an Array vs. Linked List:

In an array, binary search can be applied effectively because arrays provide direct access to elements through their index. Since arrays have contiguous memory allocation, accessing elements in constant time is possible, making binary search efficient.

However, linked lists do not provide direct access to elements using their index. In a singly linked list, to access the middle element (required for binary search), we would need to traverse the list from the head node to find the middle element, which takes $O(n/2)$ time complexity. This essentially negates the time advantage of binary search since finding the middle element would take linear time.

Employing Binary Search in a Linked List:

Despite the limitation, binary search can still be used with some adaptations for certain types of linked lists. The key is to use a doubly linked list or maintain a reference to the tail node in a singly linked list to enable backward traversal. By doing so, we can access the middle element (or any element) in $O(n/2)$ time, which is more efficient than a linear search. However, the overall time complexity of binary search in a linked list remains $O(\log n)$ for finding a target element due to the divide-and-conquer nature of binary search.

Justification for Applicability:

The binary search algorithm is still useful for certain cases where the benefits of maintaining a sorted linked list outweigh the additional overhead of accessing elements indirectly. If the linked list remains relatively small or the search operations significantly outnumber the insertions or deletions, using a sorted linked list with binary search can be reasonable.

In conclusion, binary search can be employed on data stored in a linked list, but its efficiency depends on the specific characteristics of the linked list and the frequency of search operations compared to insertions and deletions. It may not be as efficient as binary search on an array due to the lack of direct element access, but with certain adaptations, it can still be a viable option for searching in sorted linked lists.

3 (c) 5

▼ Briefly discuss inserting mechanism of an item at the beginning, after a given node, at the end and also to a sorted list #2017 #2014

Inserting Mechanisms in Different Scenarios:

1. Inserting at the Beginning:

- Create a new node with the data to be inserted.
- Set the "next" pointer of the new node to point to the current first node (head).
- Update the head pointer to point to the new node.
- If the linked list is initially empty, the new node becomes the only node, and the head points to this node.

2. Inserting after a Given Node:

- Locate the node after which the new node is to be inserted (let's call it "current_node").
- Create a new node with the data to be inserted.
- Set the "next" pointer of the new node to point to the next node of "current_node."
- Update the "next" pointer of "current_node" to point to the new node.

3. Inserting at the End:

- Create a new node with the data to be inserted.
- Traverse the linked list from the head to find the current last node (tail).
- Set the "next" pointer of the current last node to point to the new node.
- Update the tail pointer to point to the new node.
- If the linked list is initially empty, the new node becomes the first and last node, and the head points to this node.
- Update the tail or rear pointer to point to the new node.
- If the data structure is empty, the new node becomes the first and last node, and both head and tail (in the case of a doubly linked list) point to this node.

4. Inserting into a Sorted List:

- Start at the head or root of the sorted data structure and traverse through it to find the appropriate position for the new item.
- Compare the data of each node or element with the data of the new item to determine the correct insertion point.
- Create a new node or element with the data to be inserted.
- Adjust the "next" or "right" pointers to connect the new node appropriately in the sorted order.

1. Inserting at the Beginning:

To insert an item at the beginning of a data structure like a linked list or an array, you follow these steps:

- For Linked List:
 - Create a new node with the data of the item to be inserted.
 - Set the 'next' pointer of the new node to the current head of the linked list.
 - Update the head of the linked list to point to the new node.
- For Array:

- Shift all existing elements one position to the right to make space for the new item.
 - Insert the new item in the first position of the array.
1. Inserting after a Given Node:
- To insert an item after a given node in a linked list, follow these steps:
- Create a new node with the data of the item to be inserted.
 - Set the 'next' pointer of the new node to the 'next' pointer of the given node.
 - Update the 'next' pointer of the given node to point to the new node.
1. Inserting at the End:
- To insert an item at the end of a data structure like a linked list or an array, you follow these steps:
- For Linked List:
 - Create a new node with the data of the item to be inserted.
 - Traverse the linked list to find the last node.
 - Set the 'next' pointer of the last node to point to the new node.
 - For Array:
 - Insert the new item in the last position of the array.
1. Inserting into a Sorted List:
- To insert an item into a sorted list (e.g., sorted linked list, sorted array), you follow these steps:
- For Sorted Linked List:
 - Create a new node with the data of the item to be inserted.
 - Traverse the linked list to find the correct position to insert the new node while maintaining the sorting order.
 - Adjust the 'next' pointers of the nodes accordingly to insert the new node.
 - For Sorted Array:
 - Find the correct position to insert the new item while maintaining the sorting order.
 - Shift all the elements to the right, starting from the insertion position, to make space for the new item.
 - Insert the new item into the correct position.

4 (a) 2.75

▼ What is hash table? What can be the techniques to avoid collision?

Hash Table:

A hash table is a data structure that stores key-value pairs and provides efficient access to values based on their corresponding keys. It is also known as a hash map. The key is hashed using a hash function, which converts it into an index within an array.

Techniques to Avoid Collision:

1. Closed Hashing:
 - Linear Probing: If a collision occurs while inserting an element, the algorithm looks for the next available (empty) slot in the table, moving linearly through the array until it finds one.
 - Quadratic Probing: Similar to linear probing, but it uses a quadratic function to search for the next available slot, providing better clustering avoidance.

- Double Hashing: Utilizes a second hash function to calculate the step size for probing, reducing clustering and providing more uniform distribution.

2. Open Hashing:

- In separate chaining, each index in the hash table points to a linked list or another data structure that holds multiple key-value pairs that have the same hash value.
- When a collision occurs, the new key-value pair is appended to the linked list at the corresponding index.
- This technique ensures that multiple elements with the same hash value can be stored without clustering

4 (b) 3

- ▼ Draw a hash table with chaining and a size of 9. Use the hash function " $k \% 9$ " to insert the keys 5, 29, 20, 0, 27, 26, 17 and 18 into your table

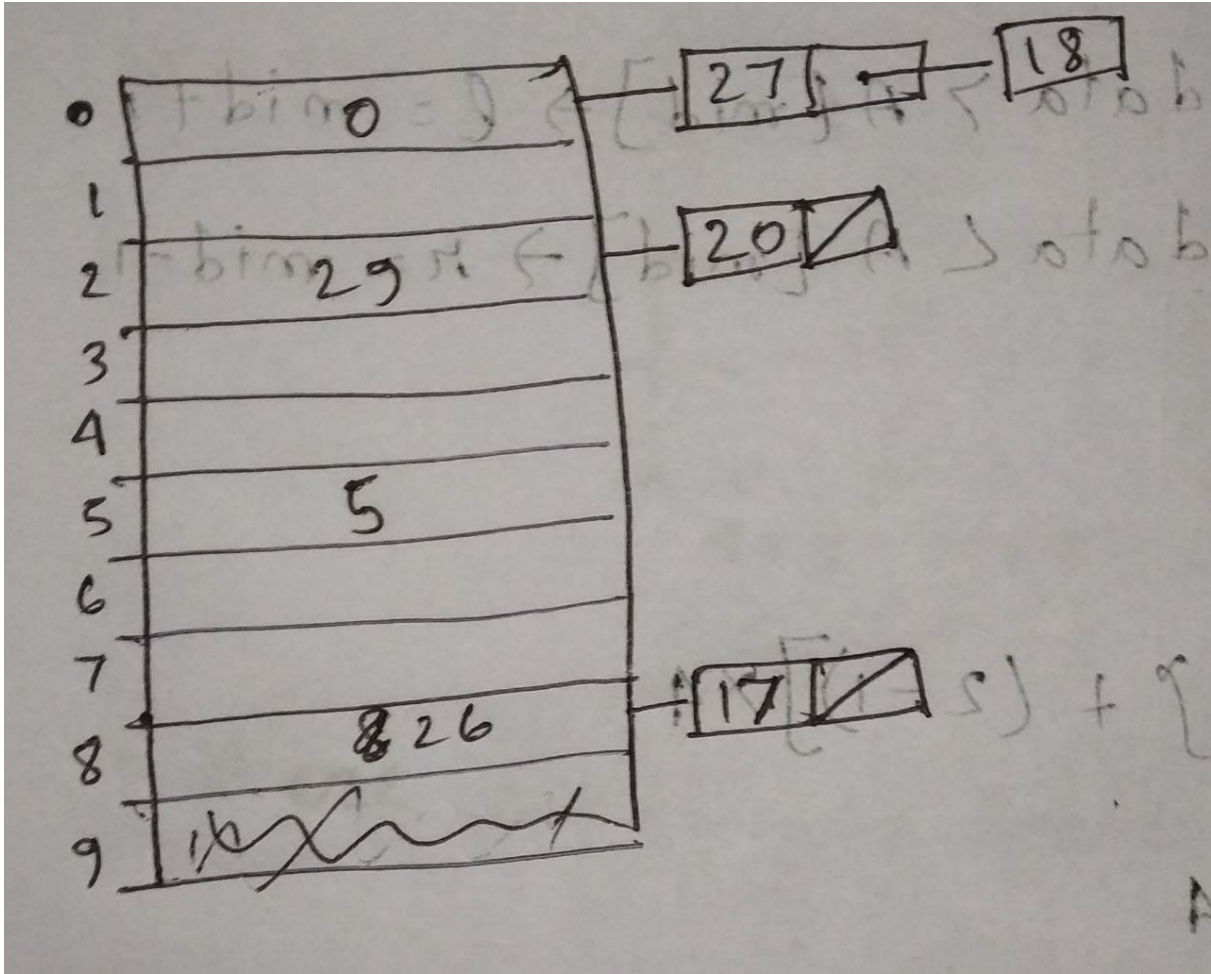
5, 29, 20, 0, 27, 26, 17, 18

no. of M words

011 | 20

key location

5	$5 \div 9 = 0$ ←
29	$29 \div 9 = 3$
20	$20 \div 9 = 2$
0	$0 \div 9 = 0$
27	$27 \div 9 = 3$
26	$26 \div 9 = 2$
17	$17 \div 9 = 1$
18	$18 \div 9 = 2$



4 (c) 3

Question is same as #2021 but table slight different

shown below:

0	1	2	3	4	5	6	7	8	9
		42	23	34	52	46	33		

▼ Answer

5 (a) 3

▼ What is a queue, how is it different from a stack and how is it implemented?

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are inserted at the rear (enqueue) and removed from the front (dequeue).

Difference between a queue and a stack:

1. FIFO vs. LIFO: A queue follows the FIFO principle, while a stack follows the LIFO (Last-In-First-Out) principle. In a stack, the last element inserted is the first to be removed, opposite to the behavior of a queue.

2. Insertion and Removal: In a queue, elements are inserted at the rear and removed from the front, whereas, in a stack, elements are inserted and removed from the top.
3. Real-life Analogies: Queues can be likened to lines at a ticket counter, where the first person in line is served first, while stacks can be likened to a stack of plates, where the last plate added is the first one taken off.

Implementation of a queue:

A queue can be implemented using arrays or linked lists. Each approach has its own advantages and considerations:

1. Array-based Queue:

- In an array-based implementation, a fixed-size array is used to store elements.
- Two pointers, front and rear, are maintained to keep track of the front and rear positions.
- Enqueue: The rear pointer is incremented to insert an element at the end.
- Dequeue: The front pointer is incremented to remove the first element.

2. Linked List-based Queue:

- In a linked list-based implementation, nodes form a chain to store elements.
- A front pointer points to the first node, and a rear pointer points to the last node.
- Enqueue: A new node is created and linked to the rear node, updating the rear pointer.
- Dequeue: The front node is removed, and the front pointer is updated to the next node in the queue.

Queue:

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, meaning that the element inserted first will be the first one to be removed. It works like a queue in real life, where the first person to join the queue is the first one to be served.

Difference Between a Queue and a Stack:

The main difference between a queue and a stack lies in the order of element removal. In a queue, the first element added (the front) is the first one to be removed. On the other hand, in a stack, the last element added (the top) is the first one to be removed, which follows the Last-In-First-Out (LIFO) principle.

Implementation of a Queue:

There are several ways to implement a queue, but two common implementations are using arrays and linked lists.

1. Array-based Implementation:

- In this approach, a fixed-size array is used to store the elements of the queue.
- We maintain two pointers, front and rear, to keep track of the first and last elements of the queue.
- When an element is enqueued, it is added to the rear of the queue, and the rear pointer is updated.
- When an element is dequeued, the front element is removed, and the front pointer is updated to the next element.
- The main advantage of this implementation is that it provides direct access to elements using indexes, but it may require resizing the array when the queue is full.

2. Linked List-based Implementation:

- In this approach, a linked list is used to represent the elements of the queue.
- We maintain a reference to both the front and rear nodes of the linked list.

- When an element is enqueued, it is added to the rear of the linked list, and the rear pointer is updated.
- When an element is dequeued, the front element is removed, and the front pointer is updated to the next element in the linked list.
- The advantage of this implementation is that it can handle dynamic sizes without resizing, but it may require more memory due to the pointers.

5 (b) 3

▼ Write the steps involved in the insertion and deletion of an element in a stack

Insertion of an Element in a Stack:

1. Check for Overflow: Before inserting an element into the stack, check whether the stack is full and there is enough space to add a new element.
2. Increment the Top Pointer: If there is space in the stack, increment the top pointer to point to the next available position in the stack.
3. Insert the Element: Place the new element at the position indicated by the top pointer.

Deletion of an Element from a Stack:

1. Check for Underflow: Before deleting an element from the stack, check whether the stack is empty (underflow condition) to avoid any invalid operation.
2. Access the Top Element: If the stack is not empty, access the element that is currently at the top of the stack.
3. Decrement the Top Pointer: After accessing the top element, decrement the top pointer to remove the element from the stack.

Insertion (Push) Operation:

1. Check for Overflow: Before inserting an element into the stack, we need to check if the stack is already full. If the stack is full, it is said to be in overflow state, and we cannot insert any more elements.
2. Increment the Top Pointer: If there is space in the stack (no overflow), we increment the top pointer to point to the next available position in the stack.
3. Insert the Element: We place the new element into the position pointed to by the top pointer. This effectively inserts the element at the top of the stack.

Deletion (Pop) Operation:

1. Check for Underflow: Before deleting an element from the stack, we need to check if the stack is empty. If the stack is empty, it is said to be in underflow state, and we cannot remove any elements.
2. Access the Top Element: If the stack is not empty (no underflow), we access the element that is currently at the top of the stack. This is the element we are going to remove.
3. Decrement the Top Pointer: After accessing the top element, we decrement the top pointer to remove the element from the stack.
4. Optional: Return or Use the Removed Element: Depending on the application, we may want to return or use the element that was removed from the stack.

5 (c) 2.75

▼ Simulate the postfix expression evaluation algorithm using **18,6,/,6,2,/,+,3,-,4,*** by showing Stack's contents as each element is scanned #2021 #2014(same but different values)

18 6 / 6 2 / + 3 - 4 *

The first character scanned is "18", which is an operand, so push it to the stack.

18	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "6", which is an operand, so push it to the stack.

6	
18	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "/", which is an operator, so pop its two operands from the stack. Pop 6 from the stack for the right operand and then pop 18 from the stack to make the left operand.

	18 / 6 = 3
Stack	Expression

Next, push the result of 18 / 6 (3) to the stack.

3	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "6", which is an operand, so push it to the stack.

6	
3	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "2", which is an operand, so push it to the stack.

2	
6	
3	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "/", which is an operator, so pop its two operands from the stack. Pop 2 from the stack for the right operand and then pop 6 from the stack to make the left operand.

3	6 / 2 = 3
Stack	Expression

Next, push the result of 6 / 2 (3) to the stack.

3 3	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "+", which is an operator, so pop its two operands from the stack. Pop 3 from the stack for the right operand and then pop 3 from the stack to make the left operand.

	3 + 3 = 6
Stack	Expression

Next, push the result of 3 + 3 (6) to the stack.

6	
Stack	Expression

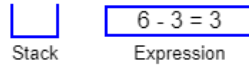
18 6 / 6 2 / + 3 - 4 *

The next character scanned is "3", which is an operand, so push it to the stack.

3 6	
Stack	Expression

18 6 / 6 2 / + 3 - 4 *

The next character scanned is "-", which is an operator, so pop its two operands from the stack. Pop 3 from the stack for the right operand and then pop 6 from the stack to make the left operand.



Next, push the result of 6 - 3 (3) to the stack.



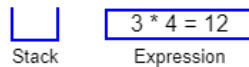
18 6 / 6 2 / + 3 - 4 *

The next character scanned is "4", which is an operand, so push it to the stack.



18 6 / 6 2 / + 3 - 4 *

The next character scanned is "*", which is an operator, so pop its two operands from the stack. Pop 4 from the stack for the right operand and then pop 3 from the stack to make the left operand.



Next, push the result of 3 * 4 (12) to the stack.



Since we are done scanning characters, the remaining element in the stack (12) becomes the result of the postfix evaluation.

Postfix notation: 18 6 / 6 2 / + 3 - 4 *
Result: 12

6 (a) 2

- ▼ When do we call a binary tree to be a 2-tree? What is the depth of a complete binary tree with 10,00,000 nodes?

A binary tree is called a 2-tree if every node in the tree has either 0 or 2 children. In other words, there are no nodes with only one child (unary nodes) in a 2-tree. Every internal node (non-leaf node) in a 2-tree has exactly two children.

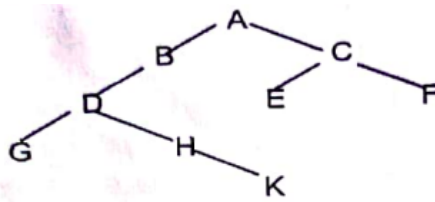
We know that, The depth of complete binary tree of n nodes will be $D_n = \log_2(n + 1)$.

Here, D_n is the height or depth of the tree and n is the number of nodes

So, $D_n = \log_2(10,00,000) \approx 20$

6 (b) 3

Traverse the following tree in preorder, inorder and postorder **#revised-1 #2018**



▼ Answer

Preorder Traversal:

In preorder traversal, we visit the root node first, then traverse the left subtree, and finally traverse the right subtree.

Preorder Traversal: A, B, D, G, H, K, C, E, F

Inorder Traversal:

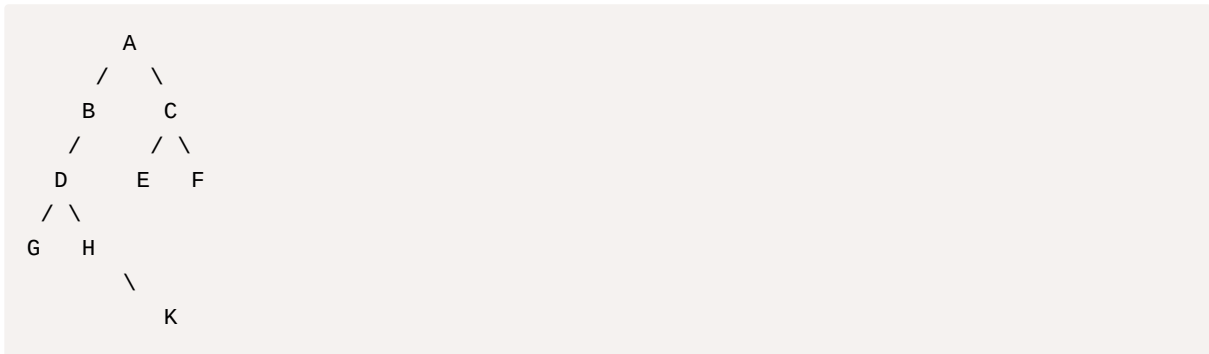
In inorder traversal, we traverse the left subtree first, then visit the root node, and finally traverse the right subtree.

Inorder Traversal: G, D, H, K, B, A, E, C, F

Postorder Traversal:

In postorder traversal, we traverse the left subtree first, then the right subtree, and finally visit the root node.

Postorder Traversal: G, K, H, D, B, E, F, C, A



6 (c) 3.75

▼ How can binary search tree overcome the disadvantages of array and linked list? Explain the process of inserting a new unique item in a binary search tree with a simple example

Binary Search Tree (BST) overcomes the disadvantages of both arrays and linked lists by providing efficient search, insertion, and deletion operations.

1. Search Efficiency:

- Array: Searching in an array takes $O(n)$ time in the worst case since we may need to traverse the entire array to find the desired element.
- Linked List: Similarly, searching in a singly linked list takes $O(n)$ time in the worst case as we may need to traverse the list from the beginning.

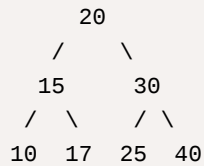
In a BST, searching is very efficient, taking $O(\log n)$ time in the average case and $O(h)$ time in the worst case. The BST property allows us to efficiently navigate to the desired node by comparing the target value with the values at each node.

2. Insertion Efficiency:

- Array: Inserting an element in an array requires shifting all the elements after the insertion point, which takes $O(n)$ time.
- Linked List: Inserting a node at the beginning or middle of a linked list takes $O(1)$ time if we have the reference to the previous node, but $O(n)$ time if we need to find the previous node first.

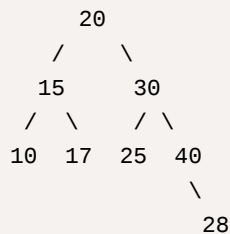
In a BST, insertion is also efficient, taking $O(\log n)$ time in the average case and $O(h)$ time in the worst case. We can find the appropriate location for the new item by comparing its value with the values at each node, and then insert it as a leaf node.

Considering the following BST:



Let's insert a new unique item with the value 28 into this BST:

1. Start at the root (20).
2. Compare 28 with 20. Since $28 > 20$, move to the right child (30).
3. Compare 28 with 30. Since $28 > 30$, move to the right child (40).
4. The right child of 40 does not exist, so insert 28 as the right child of 40.



7 (a) 4

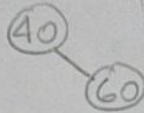
- ▼ Suppose the following eight numbers are inserted in order into an empty binary search tree:
40,60,50,33,35,55,11,12. Show the tree as each number is inserted into a binary search tree **#revised-1**

40, 60, 50, 33, 35, 55, 11, 12

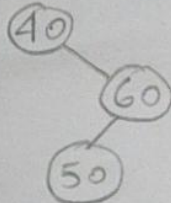
1. Inserting 40:



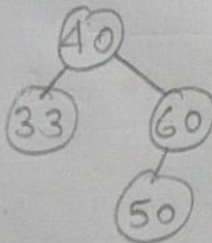
2. Inserting 60:



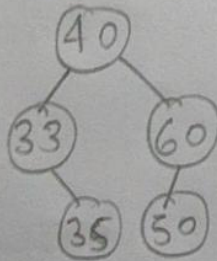
3. Inserting 50:



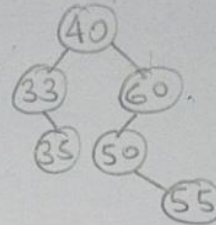
4. Inserting 33:



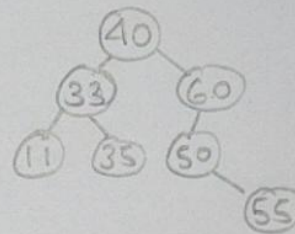
5. Inserting 35:



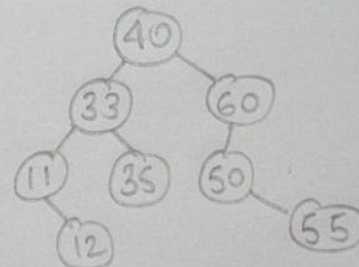
6. Inserting 55:



7. Inserting 11:



8. Inserting 12:



7 (b) 4.75

Consider the following adjacency matrix below:

$$A = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Now find out A^2, A^3, A^4, B_4 and from that make the path matrix and tell whether this is strongly connected or not #revised-1

▼ Answer

```
A^2:
>>> print(A2)
[[2 2 2 1]
 [0 2 0 2]
 [2 1 2 0]
 [0 2 0 2]]
>>>
>>> print("A^3:")
A^3:
>>> print(A3)
[[4 5 4 3]
 [0 4 0 4]
 [4 3 4 1]
 [0 4 0 4]]
>>>
>>> print("A^4:")
A^4:
>>> print(A4)
[[ 8 12  8  8]
 [ 0  8  0  8]
 [ 8  8  8  4]
 [ 0  8  0  8]]
>>> B4 = A + A2 + A3 + A4
>>>
>>> print("B_4:")
B_4:
>>> print(B4)
[[15 20 15 12]
 [ 0 15  0 15]
 [15 12 15  5]
 [ 0 15  0 15]]
```

$$A = \begin{vmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{vmatrix}$$

8 (a) 1

▼ What is directed graph? #revised-1 #2019

A directed graph, also known as a digraph, is a type of graph where edges have a direction associated with them. In a directed graph, the edges are represented as ordered pairs (u, v) , where u is the starting vertex and v is the ending vertex. The direction of the edge indicates a one-way relationship from vertex u to vertex v .

In other words, if there is a directed edge (u, v) in the graph, it means that there is a directed path from vertex u to vertex v , but not necessarily from v to u .

A directed graph, also known as a digraph, is a type of graph in which edges have a specific direction associated with them. In a directed graph, each edge is represented by an ordered pair of vertices (u, v) , where ' u ' is the source vertex, and ' v ' is the destination vertex. This directional nature of edges distinguishes directed graphs from undirected graphs, where edges have no specific direction.

8 (b) 4

- ▼ Discuss the sequential representation of graph with example [#revised-1](#) #2019

There are two commonly used sequential representations of graphs: the adjacency matrix and the adjacency list.

Adjacency Matrix:

An adjacency matrix is a 2D array of size $n \times n$, where n is the number of vertices in the graph. The matrix is initialized with all zeros, and the presence or absence of an edge between two vertices is represented by 1 or 0

Example:

Consider the following undirected graph:

```
1----2
|    |
4----3
```

The adjacency matrix for this graph is:

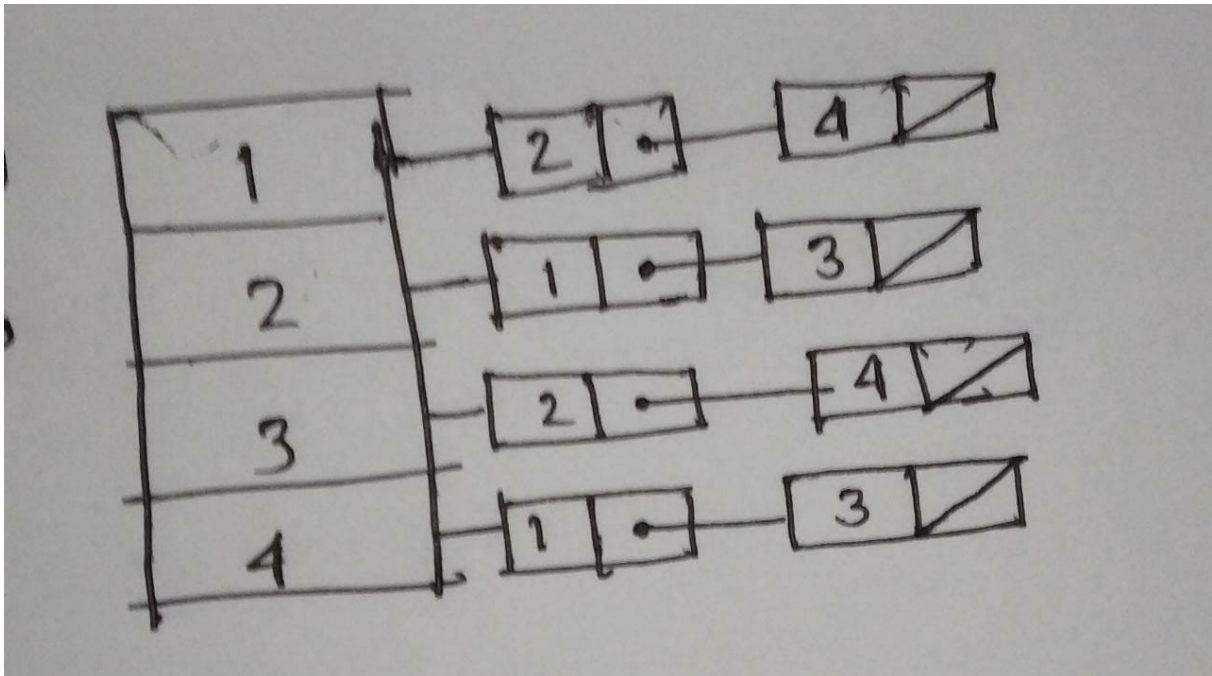
```
    | 1 2 3 4
---|-----
1 | 0 1 0 1
2 | 1 0 1 0
3 | 0 1 0 1
4 | 1 0 1 0
```

Adjacency List:

An adjacency list is a collection of linked lists or arrays where each vertex has a list containing its neighboring vertices.

Example:

Using the same graph as above, the adjacency list representation would be:



8 (c) 3.75

Use the Warshall's algorithm to find the shortest path matrix of the adjacency matrix given below:

$$A = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

▼ Answer

$$A = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

2019

1 (a) 3

▼ Define data structure. Why data structure is necessary? #revised-1 #2016

Definition of Data Structure:

A data structure is a systematic way to organize data so that it can be used efficiently

Data structures are necessary for a number of reasons:

1. Data structures allow us to store and retrieve data quickly and easily

2. Data structures help us to avoid errors when working with data. For example, a linked list can be used to store a list of items in a specific order, and it will automatically ensure that the items are always inserted and deleted in the correct location.

3. Data structures are often reusable

For example, instead of creating multiple variables of same type we can create an array and store all the values.

1 (b) 2.75

▼ What do you mean by Linear data structure and Nonlinear data structure ? Give example **#revised-1 #2016**

Linear Data Structure:

A data structure is linear when all the elements are arranged in a linear or sequential order

For example, linked List. In a linked list each node points to the next element in the sequence and it forms a linear chain

Nonlinear Data Structure:

A data structure is non linear when all the elements are not arranged in a linear or sequential order

For example, binary tree. In a binary tree each element can have up to two children. This creates a hierarchical structure

1 (c) 3

▼ What is 2d array **#revised-1?** How can you represent 2d array in memory? **#2020 #2017**

2D Array:

A **2D array** is a data structure that stores a collection of data items in a two-dimensional grid. Each data item in a 2D array is identified by two indices: a row index and a column index.

2D array can be represented in memory in two ways: row major and column major order

1. Row Major Order: In a row major order, elements of each row are stored contiguously in memory, followed by the elements of the next row and so on. This is the most common way to store 2D array in the memory

For example,

```
array={{1, 2, 3},  
       {4, 5, 6},  
       {7, 8, 9}}
```

In memory, the elements would be stored as follows,

```
1, 2, 3, 4, 5, 6, 7, 8, 9
```

2. Column Major Order: In a column major order, the elements of each column are stored contiguously in the memory, followed by the elements of the next column and so on. This is less common to store a 2D array in the memory

For example,

```
array={{1, 2, 3},  
       {4, 5, 6},  
       {7, 8, 9}}
```

In memory, the elements would be stored as follows,

```
1, 4, 7, 2, 5, 8, 3, 6, 9
```

2 (a) 3.75

Suppose Book(2:10, -4:-1, 5:8) is a 3D array with base=200, w=4, calculate Book[5,-1,7] address in a row major order and column major order **#revised-1**

2 (b) 4

- ▼ What is pointer array? How a pointer can save memory space when storing a 2D array? #2014 **#revised-1**

A **pointer array** is an array that stores pointers to other data items. Pointer arrays can be used to store pointers to any type of data, including other arrays.

How a pointer can save memory space when storing a 2D array:

A 2D array is a data structure that stores a collection of data items in a two-dimensional grid. Each data item in a 2D array is identified by two indices: a row index and a column index.

To store a 2D array in memory, we need to allocate memory for each element of the array. This can use a lot of memory, especially if the array is large.

To save memory, we can store a 2D array using a pointer array. A pointer array only needs to allocate memory for the pointers to the elements of the 2D array. This can save a lot of memory, especially if the 2D array is large.

For example,

```
public class PointerArray2DArray {

    public static void main(String[] args) {
        int rows = 3;
        int cols = 4;

        // Allocate memory for the pointer array.
        int[][] array = new int[rows][];

        // Allocate memory for each row of the array.
        for (int i = 0; i < rows; i++) {
            array[i] = new int[cols];
        }

        // Initialize the elements of the array.
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                array[i][j] = i + j;
            }
        }

        // Print the elements of the array.
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(array[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
0 1 2 3
1 2 3 4
2 3 4 5
```

2 (c) 1

▼ Is there any difference between a record and a linear array? **#revised-1 #2015**

Yes, there is a difference between a record and a linear array.

A **record** is a data structure that stores a collection of data items of different types. Each data item in a record is identified by a name.

A **linear array** is a data structure that stores a collection of data items of the same type. Each data item in a linear array is identified by an index.

3 (a) 2.75

What are the advantages of Linked List over array?

3 (b) 3

Briefly discuss inserting mechanism of an item at the beginning, after a given node and at the end.

3 (c) 3

What is two way lists? Why it is important? Explain with Schematic diagram

4 (a) 2.75

Briefly discuss hash collision of resolution techniques

4 (b) 3

Draw a hash table with open addressing and a size of 9. Use the hash function " $k \% 9$ " and quadratic probing. Insert the keys: 5, 29, 20, 0, 27, 26, 17 and 18 into your table

4 (c) 3

Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function $x \bmod 10$, which of the following statements are true? explain

1. 9679, 1989, 4199 hashes to the same value
2. 1471, 6171 has the same value
3. All elements hash to the same value
4. Each element hashes to a different value
5. None of the above

5 (a) 1.75

Differentiate between stack and queue

5 (b) 3

Convert the following infix expression to its equivalent prefix and postfix expression. $A + (B * C / 5 - (D / E \uparrow F) * G) * H$

5 (c) 3

Simulate the postfix expression evaluation algorithm using 18, 6, /, 6, 2, +, *, 12, 4, /, - by showing Stack's contents as each element is scanned

5 (d) 1

What is dequeue?

6 (a) 2

▼ What is complete binary tree? What is the parent child relationship? **#revised-1 #2015**

A **complete binary tree** is a binary tree in which every node, except possibly the leaves, has two children.

Parent-child relationship in a complete binary tree:

In a complete binary tree, each node has at most two children, a left child and a right child. The parent of a node is the node that is one level above it in the tree.

6 (b) 3

Discuss the linked representation of binary tree in memory

6 (c) 3.75

Simulate the maxheap algorithm for the following values: 100, 29, 90, 148, 12, 34, 90, 89, 120, 99 and 12

7 (a) 2

What is Directed Graph? Explain #2020

7 (b) 2

Briefly discuss the advantages of binary search tree over array and linked list

7 (c) 4.75

Use the Warshall's algorithm to find the shortest path matrix of the weighted matrix given below

```
W = | 2 5 0 0 |
     | 2 0 1 4 |
     | 5 3 3 3 |
     | 4 2 3 0 |
```

8 (a) 3

Discuss the sequential Representation of Graph with example #2020

8 (b) 2.75

- ▼ How many ways a graph G can be traversed? What is the significance of the STATUS field? #revised-1

Ways to Traverse a Graph:

A graph G can be traversed in two fundamental ways: Depth-First Search (DFS) and Breadth-First Search (BFS).

1. **Depth-First Search (DFS):** In DFS, the traversal starts from a selected source node and explores as far as possible along each branch before backtracking. It involves visiting a node, marking it as visited, and then recursively visiting its adjacent unvisited nodes. DFS is implemented using a stack. It is useful for tasks like finding paths, cycles, and connected components in a graph.
2. **Breadth-First Search (BFS):** In BFS, the traversal starts from a selected source node and explores all of its adjacent nodes before moving on to their adjacent nodes. BFS is implemented using a queue. It is suitable for finding the shortest path and analyzing the connectivity of a graph.

Significance of the STATUS Field:

The STATUS field is like a tag used in graph traversal, especially in BFS. It helps keep track of nodes as we explore the graph. It can be in different states, like:

1. **Unvisited (Not Processed):** This status indicates that the node has not been visited or processed yet. When the traversal algorithm starts, all nodes typically begin in this state.
2. **Visited (Processed):** This status indicates that the node has been visited and processed by the traversal algorithm. Once a node is visited, it is marked as visited to avoid processing it multiple times.
3. **Queued (Enqueued):** In the BFS algorithm, this status is used to mark nodes that have been added to the queue for further processing. It helps prevent duplicate enqueueing.

8 (c) 3

Consider the adjacency list of the Graph G in the following table. Find the nodes that are reachable from node C using Depth First Search

Node	Adjacency	Node	Adjacency
A	G, E	E	C
B	C	F	A, B
C	F	G	B, C, E
D	C	H	D

2018

1 (a) 1.75

- ▼ How can you sort the elements of a matrix? Explain #revised-1

We can sort a matrix by treating it as a 1D array. For example, considering a 2D array

5	4	7
1	3	8
2	9	6

In row major order the above matrix in memory will be as, 5,4,7,1,3,8,2,9,6

now we can sort this matrix using selection sort.

In selection sort there will be two partition: sorted and unsorted

The steps will be,

1. Find the minimum element in the unsorted partition
2. Swap the minimum element with the element in the sorted partition
3. Repeat steps 1 and 2 until the array is sorted
4. Convert the 1D array back to 2D array

Hence, the elements of a matrix is sorted

1 (b) 4

How does a pointer array can save memory when store a variable sized group of data ? Discuss with necessary figures #2015

1 (c) 3

Maze(2:6, -4:1, 1:5) is a 3D array with base=100, w=4. Calculated Maze [5, -1, 4] address in a row major order and column major order **#revised-1**

2 (a) 3

What is linked list? Discuss different types of linked list with diagram

2 (b) 4

Discuss the advantages and disadvantages of linked list

2 (c) 1.75

What is Garbage collection? When does it take place?

3 (a) 3

What is sparse matrix? How can you locate element a_{ij} of a sparse matrix from a 1D array? #2020 #2017

3 (b) 1.75

Discuss the array representation mechanism of stack

3 (c) 4

Convert the following infix expression to its equivalent prefix and postfix expression

1. $A+(B-C+D)*E/F+G\uparrow H$
2. $1+2-(3*4/5\uparrow 6)*7$

4 (a) 1.75

What is polish notation? What are the benefits of polish notation?

4 (b) 4

Simulate the infix to postfix transformation algorithm for Q: $A+(B*C-(D/E\uparrow F)*G)*H$

4 (c) 3

Suppose 15 elements are maintained by array and another 15 are by Linked List. Which methods take longer time to access 9th element. Justify your answer

5 (a) 3

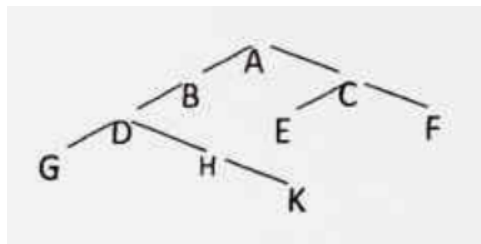
Define the following terms: siblings, successor, and level

5 (b) 2.75

What is binary search tree. How can binary search tree overcome the disadvantages of array and linked list?

5 (c) 3

Traverse the following tree in preorder, inorder and postorder #2020



6 (a) 2.75

Discuss the linked representation of Graph with example (adjacency list and matrix)

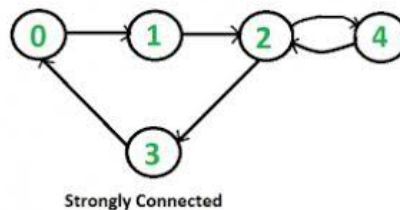
6 (b) 3

▼ Define the following terms with example: Strongly Connected Graph, Directed Graph, Weighted Graph

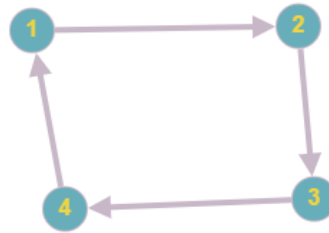
#revised-1

Strongly Connected Graph:

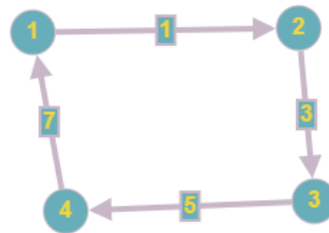
A strongly connected graph is a directed graph in which there is a directed path between any pair of vertices.



Directed Graph: A directed graph, also known as diagraph, is a type of graph where edges have direction associated with them. In the directed graph, the edges are represented as ordered pairs (u,v) , where u is the starting vertex and v is the end vertex



Weighted Graph: Weighted graph is a type of graph where edge is assigned a numerical value called weight.



6 (c) 3

Consider the following adjacency matrix below: #2016

$$A = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Now find out A2, A3, A4, B4 and from that make the path matrix and tell whether this is strongly connected or not

7 (a) 1.75

What is heap? What are the advantages of heap?

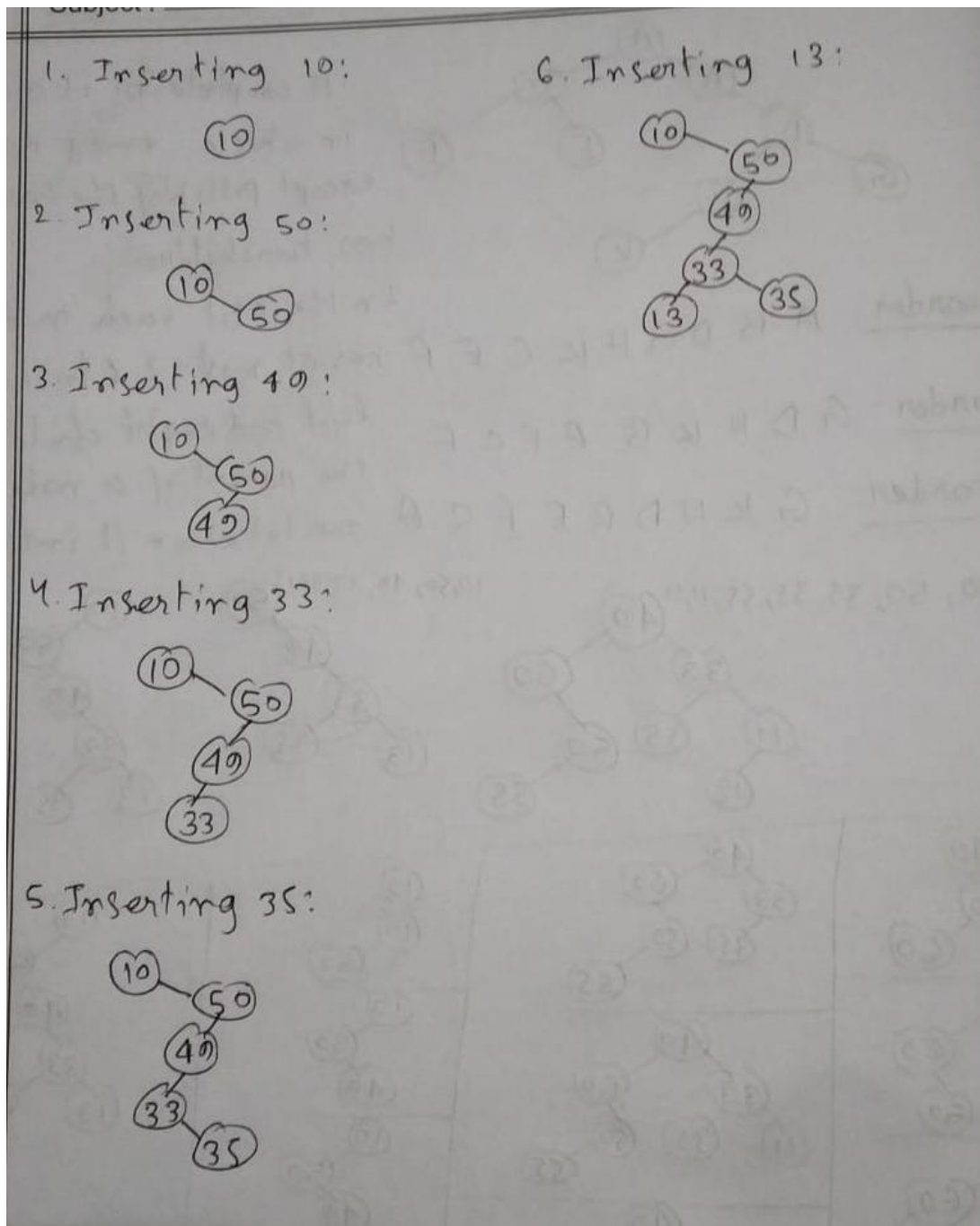
7 (b) 4

Use the Warshall's algorithm to find the shortest path matrix of the weighted matrix given below:

$$W = \begin{bmatrix} 2 & 4 & 1 & 0 \\ 3 & 0 & 0 & 5 \\ 5 & 1 & 3 & 6 \\ 3 & 2 & 3 & 0 \end{bmatrix}$$

7 (c) 3

- ▼ Suppose the following six numbers are inserted in order into an empty binary search tree. 10, 50, 49, 33, 35, 13. Show the tree as each number is inserted into a binary search tree **#revised-1**



8 (a) 3

Define hash function and hash table and load factor

8 (b) 2.75

Briefly discuss open addressing quadratic probing method with example

8 (c) 3

Draw a hash table with chaining and a size of 9. Use the hash function " $k\%9$ " to insert the keys 5, 29, 20, 0, 27, 26, 17 and 18 into your table

2017

1 (a) 2

- ▼ What is linear array? How can you find the no. of elements in any linear array? **#revised-1**

A linear array is a data structure that stores a collection of data of the same type. Each data item in a linear array is identified by an index.

To find the number of elements in a linear array, we can use the following steps:

1. Get the length of the array.
2. Return the length of the array.

```
public class FindNumberOfElements {
    public static int findNumberOfElements(int[] array) {
        return array.length;
    }

    public static void main(String[] args) {
        int[] array = {1, 2, 3, 4, 5};

        int numberOfElements = findNumberOfElements(array);

        System.out.println(numberOfElements);
    }
}
```

1 (b) 3.75

Simulate the binary search algorithm on the following data: 11 22 33 44 55 66 77 88 99 110 121 132

1 (c) 3

- ▼ What is 2D array? How can you represent 2D array in memory? **#2019**

2D array is a data structure that stores a collection of data in a two dimensional grid. Each data item stored in a 2D array is identified by two indices: a row index and a column index

2 (a) 3

Room (1:8, -4:1, 6:10) is a 3D array with base=400, w=2, calculate Room {4, -2, 7} address in a row major order and column major order **#revised-1**

2 (b) 2.75

What is sparse matrix? How can you locate element a_{ij} of a sparse matrix from a 1D array? **#2020 #2018**

2 (c) 3

What is memory saving if we store a sparse matrix in a 1D array rather than a 2D array? #2021

3 (a) 1

What is a Linked List ? Discuss with example

3 (b) 1.75

Define overflow and underflow? How can you handle them?

3 (c) 5

Briefly discuss inserting mechanism of an item at the beginning, after a given node, at the end and to a sorted list
#2020 #2014

3 (d) 1

What is Head Linked List?

4 (a) 1.75

Differentiate between stack and queue

4 (b) 4

Convert the following infix expression to its equivalent prefix and postfix expression

1. $A + (B - C) * D / E + F / G \uparrow H$
2. $(1 + 2) * 3 - 4 / 5 * 6 \uparrow 7$

4 (c) 3

Simulate the postfix expression evaluation algorithm using 20, 5, 2, *, /, 2, 3, ↑, ↑, 4, /, - by showing Stack's contents as each element is scanned

5 (a) 2

- ▼ What is complete binary tree? A complete binary tree has 1129 node, find out the depth of the tree **#revised-1**

A complete binary tree is a tree in which every node, except possibly the leaves, has two children

To find the depth of a complete binary tree with n nodes, we can use the following formula:

$$depth = \text{floor}(\log_2(n + 1))$$

Therefore, the depth of a complete binary tree with 1129 nodes is:

$$depth = \text{floor}(\log_2(1129 + 1)) = \text{floor}(\log_2(1130)) = 10$$

5 (b) 2.75

Discuss the linked representation of binary tree in memory

5 (c) 4

Simulate the maxheap algorithm for the following values: 67, 29, 90, 48, 12, 34, 90, 9, and 12

6 (a) 2

Illustrate similar and copies with example

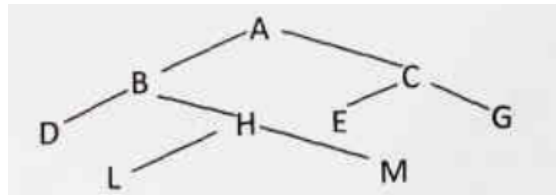
6 (b) 1

▼ What is an extended tree? **#revised-1**

Extended tree is a binary tree in which every node will have either 0 child or 2 children. Nodes with 0 number of child called external nodes and nodes with 2 children are called internal nodes

6 (c) 5.75

Simulate the postorder traversal algorithm for the following tree **#revised-1**



▼ Answer

Postorder: D L M H B E G C A

7 (a) 3

Define the following terms: Connected Graph, Path and Weighted Graph **#revised-1**

7 (b) 2

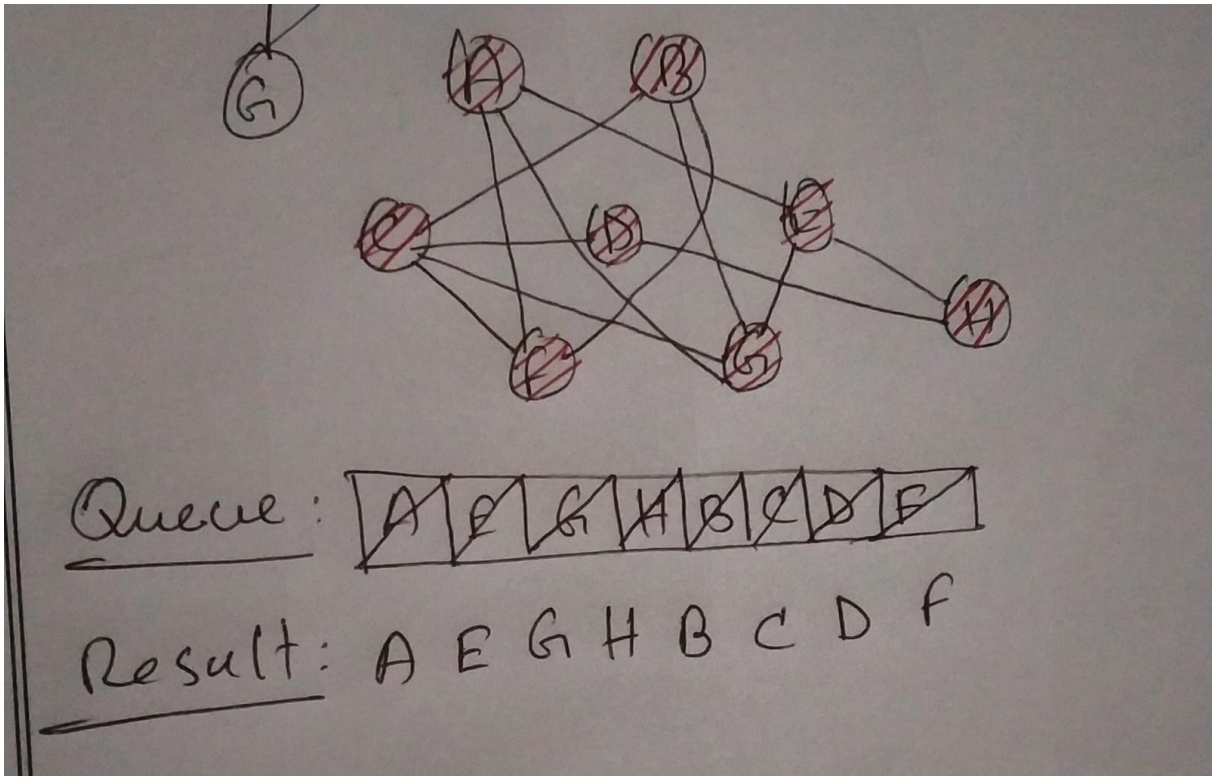
How many ways a graph G can be traversed? What is the significance of the STATUS field? **#revised-1**

7 (c) 3.75

Consider the adjacency list of the Graph G in the following table. Draw the graph and find out the path from A to F with minimum number of nodes along that path using Breadth First Search **#revised-1 #2015**

Node	Adjacency	Node	Adjacency
A	E, G	E	H
B	C	F	A, B
C	F	G	B, C, E
D	C	H	D

▼ Answer



8 (a) 2

Define deque and priority queue with example

8 (b) 4

- ▼ Write the steps of preorder and postorder traversal of a binary tree **#revised-1**

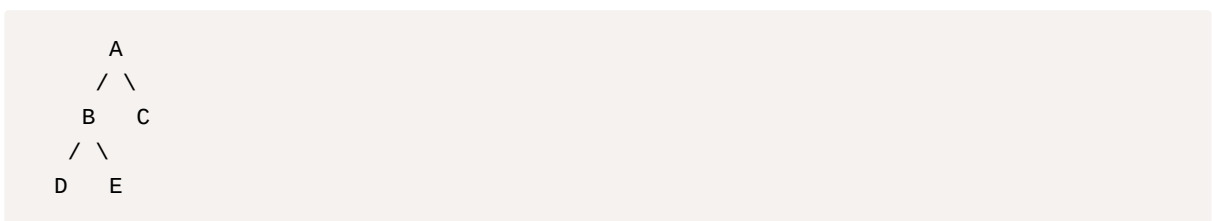
Preorder traversal:

1. Visit the root node.
2. Recursively traverse the left subtree.
3. Recursively traverse the right subtree.

Postorder traversal:

1. Recursively traverse the left subtree.
2. Recursively traverse the right subtree.
3. Visit the root node.

For example,



Preorder traversal: A B D E C

Postorder traversal: D E B C A

8 (c) 2.75

- ▼ What is binary search tree? Mention the advantages of a binary search tree **#revised-1**

A binary search tree (BST) is a data structure that stores data in a sorted order. It is a self-balancing tree, which means that it automatically maintains its sorted order as data is inserted or deleted.

Advantages:

1. Easy to implement and can be implemented in a variety of programming languages
2. Insertion and deletion operations can be performed efficiently in $O(\log n)$ time.
3. automatically maintain their sorted order as data is inserted and deleted

2016

1 (a) 3

Define data structure. Why is data structure necessary? **#2019**

1 (b) 2.75

- ▼ What do you mean by Linear data structure and Nonlinear data structure ? Give example **#2019**

A data structure is linear when all the elements are arranged in a linear or sequential order

For example, in a linked list, each node points to the next element of the sequence and form a linear chain

A data structure is non linear when all the elements are not arranged in a linear or sequential order.

For example, in a binary tree each element consists of up to two children and form a hierarchical structure

1 (c) 3

- ▼ What is sparse matrix? What is the difference between triangular matrix and tridiagonal matrix **#2020**

2 (a) 4

What are advantages of Linked List?

2 (b) 2

Suppose 10 elements are maintained by array and another 10 are by Linked List. Which methods take longer time to access 7th element. Justify your answer

2 (c) 2.75

What is two way lists? Why is it important? Explain with schematic diagram

3 (a) 3

Discuss the array representation mechanism of stack

3 (b) 2

What is polish notation? What are the benefits of polish notation?

3 (c) 3.75

Convert the following infix expression to its equivalent prefix and postfix expression

1. $A*B/C+D\uparrow(E-F*G)$
2. $1+2*3/4\uparrow5*6-7*8$

4 (a) 3

Simulate the postfix expression evaluation algorithm using 12, 6, /, 6, 2, +, *, 12, 4, /, - by showing Stack's contents as each element is scanned

4 (b) 3

What is recursion? Explain the use of recursion

4 (c) 2.75

Explain the operations on queue with example

5 (a) 4.75

Briefly discuss inserting and deleting mechanism of an item in the linked list

5 (b) 2

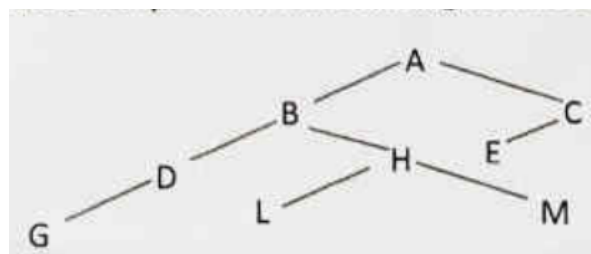
Define deque and priority queue with example

5 (c) 2

What is garbage collection? When does it take place?

6 (a) 5

Simulate the preorder traversal algorithm for the following tree **#revised-1**



▼ Answer

Preorder: A B D G H L M C E

6 (b) 2

▼ What is binary search tree? Why binary search tree is important? **#revised-1**

Binary Search Tree is a data structure that stores data in sorted order. It is self-balancing tree, which means it automatically maintain its sorted order as data is inserted or deleted.

Binary Search Tree is important because they offer a number of advantages over other data structures over storing and retrieving sorted data. They are,

1. Easy to implement and can be implemented in a variety of programming languages
2. Insertion and deletion can be performed efficiently in $O(\log n)$ time
3. Automatically maintain its sorted order as data is inserted or deleted

6 (c) 1.75

What is the difference between maxheap and minheap?

7 (a) 2.75

Define weighted graph and directed graph with example

7 (b) 3

Consider the following adjacency matrix below: #2018

```
A = | 1 0 1 1 |
    | 0 1 0 0 |
    | 0 0 1 1 |
    | 1 1 0 0 |
```

Now find out A^2 , A^3 , A^4 , B^4 and from that make the path matrix and tell whether this is strongly connected or not

7 (c) 3

Use the Warhall's algorithm to find the shorted path matrix of the weighted matrix given below

```
W = | 6 8 0 0 |
    | 3 0 0 9 |
    | 5 8 3 6 |
    | 6 2 3 0 |
```

8 (a) 4

Discuss the sequential representation of Graph with example

8 (b) 4.75

Consider the adjacency list of the Graph G in the following table. Find the nodes that are reachable from node C using Depth First Search

Node	Adjacency	Node	Adjacency
A	G, E	E	C
B	C	F	A, B
C	F	G	B, C, E
D	C	H	D

2015

1 (a) 3

- ▼ Why data structure is needed? Differentiate between linear and nonlinear data structure #2020

Data structure is needed for a number of reasons:

1. Data structure allows to store and retrieve data quickly and easily
2. Data structure helps to avoid error when working with data. For example, a linked list can be used to store a list of items and it ensures that the data is inserted and deleted in the correct location
3. Data structure is often reusable

Linear Data Structure	Non Linear Data Structure
A data structure is linear when all the elements are arranged in a linear or sequential order	A data structure is non linear when all the elements are not arranged in a linear or sequential order
Examples: Arrays, Linked List, Queues, Stacks	Examples: Tree, Graphs
Traversal is straightforward and follows a linear order	Traversal can be complex and may involve recursive methods
Access for elements is usually constant or $O(1)$ in most cases	Access for elements vary and can be $O(\log n)$ or $O(n)$, depending on the the specific structure
Each elements has a unique predecessor and successor, except for the first and last elements	Each elements can have multiple predecessor and successors

1 (b) 2

- ▼ Explain linear array representation in memory #revised-1

A linear array is represented in memory as a contiguous block of memory. Each element in the array is stored in a consecutive memory location. The first element of the array is stored at the base address of the array. The base address of an array is the memory address of the first element of the array.

The following diagram shows how a linear array is represented in a memory:

```
Base address of the array: 1000
Element 0: 10
Element 1: 20
Element 2: 30
Element 3: 40
Element 4: 50
```

The size of each element in the array depends on the data type of the array. For example, if the array is an array of integers, then each element in the array will be 4 bytes in size.

1 (c) 3.75

- ▼ Simulate the binary search algorithm on the following data: 10,20,50,70,80,90,100,110 (suppose we search for item 110)

2 (a) 3

- ▼ For column major order find out the address of the element score [15,3] from a 20X5 matrix array score with base value 100 and w=4 #revised-1

2 (b) 3.75

- ▼ How does a pointer array can save memory when stores a variable sized group of data? Discuss with necessary figures #2018

2 (c) 2

- ▼ What is a record? What is difference between a record and linear array? #2019

A record is a data structure that stores a collection of data of different types. Each data item stored in a record can be identified by a name. For example, student records, employee records

A linear array is a data structure the stores a collection of data of the same type. Each data item in a linear array can be identified by using an index. For example, list of numbers, string of characters

3 (a) 3

- ▼ Explain the representation of linked lists in memory

3 (b) 3

- ▼ Discuss header linked list. Describe grounded and circular header list

3 (c) 2.75

- ▼ Briefly explain two-way linked list

4 (a) 3

- ▼ What is stack? What are the operations on stack? Explain with example

4 (b) 3.75

Convert the following infix expression to its equivalent prefix and postfix expression #2014(same but different values)

- ▼ $A+B*C/D-E+(F/G+H\uparrow K)$

- ▼ $(1+2)\uparrow 3/4*5+7-8\uparrow 9$

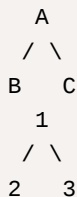
4 (c) 2

- ▼ What is priority queue? Why is it important?

5 (a) 3

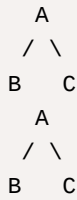
- ▼ Illustrate similar and copies of a tree with examples **#revised-1**

Similar trees are trees that have the same structure, but different values at the nodes. For example,



Both trees have the same structure, with a root node and two child nodes. However, the values at the nodes are different.

Copies of a tree are trees that have the same structure and the same values at the nodes. For example,



5 (b) 2

- ▼ What is complete binary tree? What is the parent-child relationship? #2019

5 (c) 3.75

- ▼ For the expression: $* + a - bc * - de - / fgh$ draw the tree and perform inorder and postorder traversal

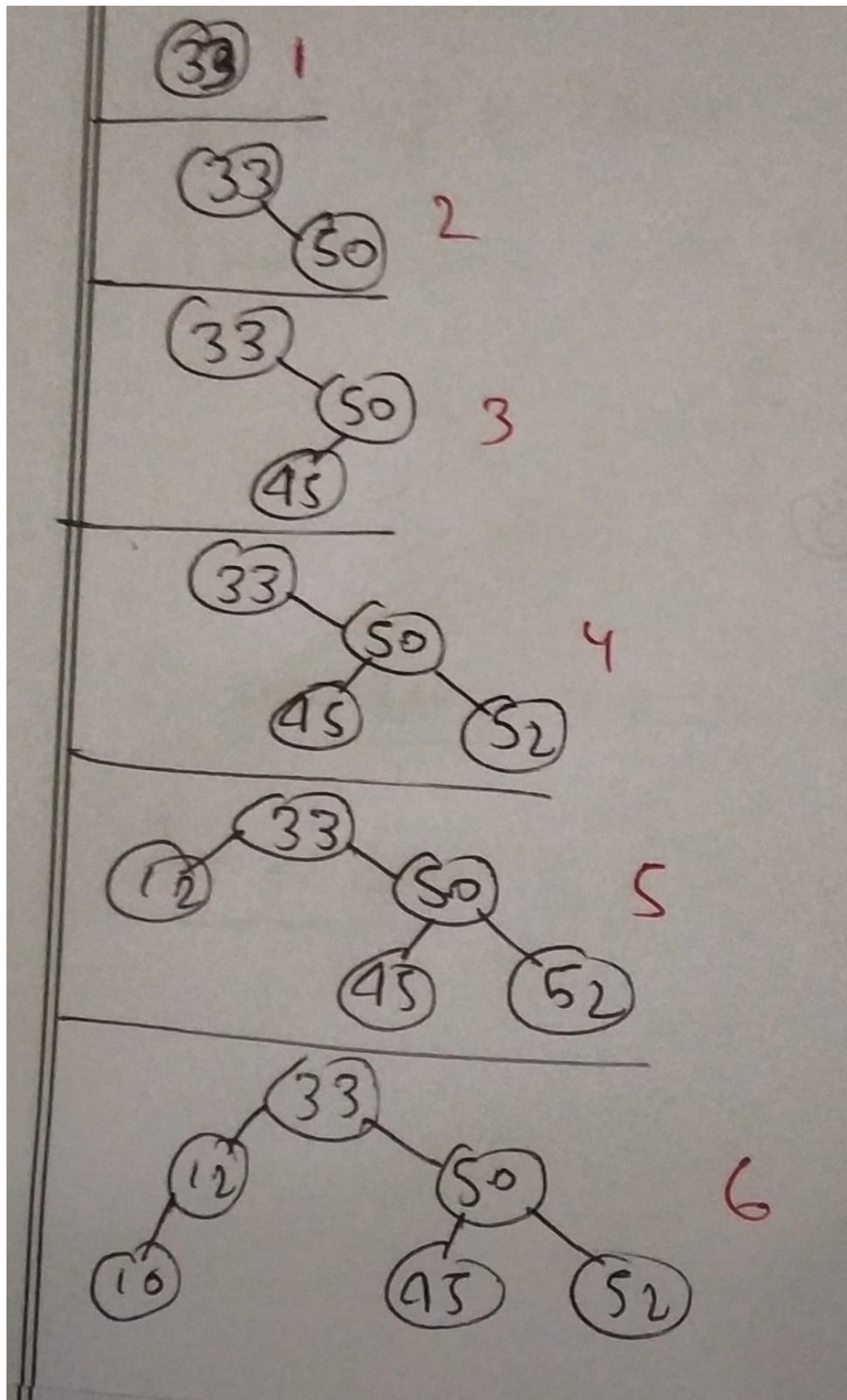
6 (a) 1.75

- ▼ What is binary search tree? #revised-1

A binary search tree is a data structure that stores data in sorted order. It is a self-balancing tree, which means it automatically maintain its sorted order as the data is inserted or deleted

6 (b) 4

- ▼ Suppose the following six numbers are inserted into an empty binary search tree: 33,50,45,52,12,10. Show the tree as each number is inserted into a binary search tree #revised-1



6 (c) 3

- ▼ Simulate the maxheap algorithm for the following values: 77,40,90,65,20,35,95,10,15

7(a) 3.75

- ▼ Define the following terms: (i) Degree of a node, (ii) Isolated node, (iii) Path, (iv) Multi Graph. #revised-1

Node Degree: The degree of a node in a graph is the number of edges incident to that node. In simpler terms, it represents how many connections or links a particular node has with other nodes in the graph. For undirected graphs, the degree counts both incoming and outgoing edges. In directed graphs, the degree is divided into in-degree (number of incoming edges) and out-degree (number of outgoing edges).

Isolated Node: An isolated node, also known as a pendant node, is a node in a graph that has a degree of zero. This means the node is not connected to any other node in the graph through an edge.

Path: In graph theory, a path is a sequence of nodes in which each node is directly connected to the next node in the sequence by an edge.

Multigraph: A multigraph is a type of graph that allows multiple edges (connections) between the same pair of nodes.

7 (b) 5

Consider the following adjacency matrix below: #2018 #2016 (but different matrix)

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

Now find out A2, A3, A4, B4 and from that make the path matrix and tell whether this graph is strongly connected or not

- ▼ Answer

8 (a) 3.75

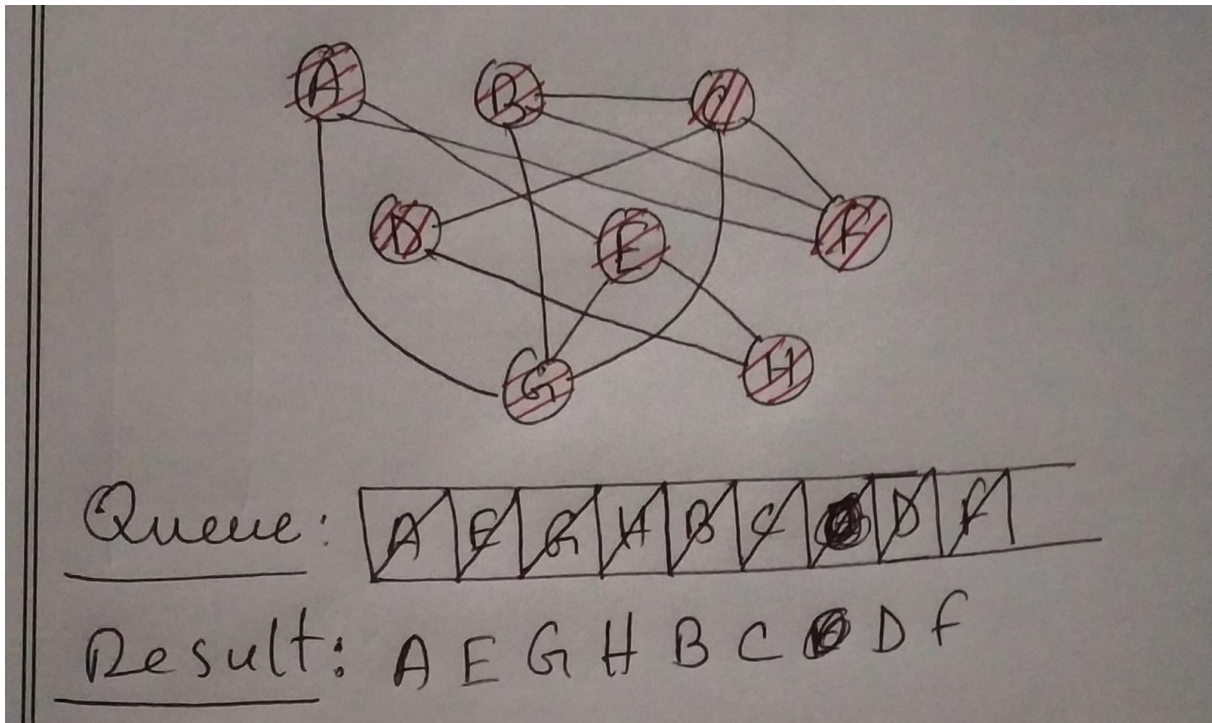
- ▼ Discuss the linked representation of Graph with example

8 (b) 5

Consider the adjacency list of the Graph G in the following table. Draw the graph and find out the path from A to H with minimum number of nodes along that path using Breadth First Search #2017

Node	Adjacency	Node	Adjacency
A	E,G	E	H
B	C	F	A, B
C	F	G	B, C, E
D	C	H	D

▼ Answer



2014

1 (a) #2015

1 (b) 3.5

▼ Explain the depth-first search technique

1 (c) 2.75

▼ What do you mean by internal and external sorting? List some names of each type #revised-1

Internal sorting: If the input data is such that it can be adjusted in the main memory at once, it is called internal sorting.

Example:

1. Bubble Sort
2. Insertion Sort
3. Quick Sort
4. Heap Sort
5. Radix Sort
6. Selection sort

External sorting: If the input data is such that it cannot be adjusted in the memory entirely at once,

Example: Merger Sort

2 (a) 2

- ▼ What is linked list? Discuss with example

2 (b) 2

- ▼ Suppose 20 elements are maintained by array and another 10 are by Linked List. Which method will take longer time to access 9th element? Justify your answer

2 (c) 4.75

Briefly discuss inserting mechanism of an item at the beginning after a given node and at the end #2020

3 (a) 2

- ▼ What is polish notation? What are the benefits of polish notation?

3 (b) 4

Convert the following infix expression to its equivalent prefix and postfix expression: #2015(same but different values)

- ▼ $A*B+(C*D/E)*F+(G\uparrow H)$

- ▼ $1*2/3+(4-5\uparrow 6)+7-8$

3 (c) 2.75

- ▼ Simulate the postfix expression evaluation algorithm using 12,6,/,6,2,+,*,12,4,/, - by showing Stack's contents as each element is scanned #2021 #2020(same but different values)

4 (a) 3.25

- ▼ What is overflow and underflow? How can you handle them?

4 (b) 3.5

- ▼ What is a two way list? Why is it important? Explain with schematic diagram

4 (c) 2

- ▼ What is garbage collection and compaction?

5 (a) 2

- ▼ For column major order find out the address of the element marks [1,2,3] from a 25X4 matrix array marks with base value 250 and w=8

5 (b) 1.75

- ▼ What is pointer and pointer array? **#revised-1**

A pointer is a variable that stores the address of another variable.

A pointer array is an array that store pointers to the data items. Pointer array can be used to store pointers to different data types, including other arrays

5 (c) 3

- ▼ How a pointer can save memory space to store a 2D array? **#revised-1** #2019

A pointer can save memory space to store a 2D array by only allocating memory for the pointers to the elements of the 2D array, and not the elements themselves. This is because pointers are very small in size, typically just the size of a memory address.

5 (d) 2

- ▼ What is the difference between Triangular matrix and Tridiagonal matrix?

6 (a) 3.75

- ▼ Define heap, leaf and depth of tree. For 5009 node, find out the depth of the tree **#revised-1**

Heap

A heap is a complete binary tree in which the value of each node is greater than or equal to the value of its child nodes.

Leaf

Node with no children is called leaf node

Depth of a tree

Maximum level of any node is the depth of a tree

$$\text{depth} = \text{floor}(\log_2(n + 1))$$

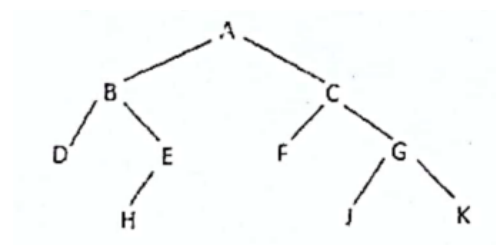
$$\text{depth} = \text{floor}(\log_2(5009 + 1)) = \text{floor}(\log_2(5010)) = 13$$

6 (b) 2

- ▼ Discuss the linked representation of binary tree in memory

6 (c) 3

Simulate the preorder traversal algorithm for the following tree av



- ▼ Answer

Preorder: A B D E H C F G J K

7 (a) 2

- ▼ Define the following graph terms: (i) Adjacent Nodes, (ii) Cycle, (iii) Connected graph, and (iv) Weighted graph **#revised-1**

(i) Adjacent Nodes:

Adjacent nodes refer to two nodes in a graph that are connected by an edge. In other words, if there is an edge

between nodes 'u' and 'v', then 'u' and 'v' are adjacent to each other.

(ii) Cycle:

A cycle in a graph is a path of edges and vertices that starts and ends at the same vertex

(iii) Connected Graph:

A connected graph is a graph in which there is a path between every pair of vertices.

(iv) Weighted Graph: A type of graph where edges are assigned a numerical value called weight

7 (b) 2.75

▼ Discuss the sequential Representation of Graph with example

7 (c) 4

Consider the following adjacency matrix below:

$$A = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

Now find out A^2, A^3, A^4, B_4 and from that make the path matrix and tell whether this is strongly connected or not

▼ Answer

8 (a)

▼ What is Directed Graph? Explain

8 (b)

▼ How many ways a graph G can be traversed? What is the significance of the STATUS field?

8 (c)

Consider the adjacency list of the Graph G in the following table. Find the nodes that are reachable from node C using Depth First Search

Node	Adjacency	Node	Adjacency
A	G, E	E	C
B	C	F	A, B
C	F	G	B, C, E
D	C	H	D

▼ Answer